

Contents

SPECIFICATION — GPU-Quicksort for Metal (parallel sorting, Swift library + CLI, Swift 6 / Metal on Apple silicon)		2
0. Intent and purpose		2
1. Actors and goals		3
2. Requirements (intent, high level)		4
2.1 Sorting semantics		4
2.2 Phase one		4
2.3 Phase two		5
2.4 Parameters, API, CLI, and cross-cutting		5
2.5 Optional items		6
3. Behavior and state model		7
3.1 Lifecycle of one sort call		7
3.2 One phase-one iteration		9
3.3 Buffers		9
4. Interfaces / contracts		9
C-01 Public API		9
C-02 SortReport		10
C-03 Parameters and device limits		11
C-04 Key encoding (order-preserving map to UInt32)		11
C-05 Phase-one descriptors (host <code>gqsort_partition</code> / <code>gqsort_fill</code>)		12
C-06 Phase-two descriptors (host <code>lqsort</code>)		12
C-07 Errors		12
C-08 Distributions ([P §5.3])		13
C-09 Package layout and shader delivery		13
C-10 Tuned-parameter table		14
C-11 CPU baselines (C/C++ shim)		14
5. Interface specification		15
5.1 Library		15
5.2 CLI <code>gpuqsort</code>		15
5.3 Cross-cutting interface contracts		17
6. Invariants (must hold in every valid implementation)		18
7. Constraints (precise and measurable)		18
7.1 Bookkeeping bound (K-09)		19
7.2 CLI exit codes (K-12)		20
8. Edge cases and failure semantics		20
9. Acceptance criteria, tests, and evals		21
9.1 Correctness (deterministic, GPU)		22
9.2 Structure and memory model (deterministic, GPU + instrumentation)		23
9.3 API, validation and errors		24
9.4 Generators and CLI (integration)		24
9.5 Performance (recorded)		26
10. Dependencies and environment		27
11. Traceability matrix (id → where realized)		27
12. Open questions and decisions to confirm		29
Appendix A — Algorithm and complexity, restated (informative)		32
A.1 Notation		32
A.2 The whole sort		33
A.3 Phase one: many threadgroups per sequence		33
A.4 Phase two: one threadgroup per sequence		34
A.5 Pivot rules		34
A.6 Where this design departs from the paper		34
A.7 Complexity		36

SPECIFICATION — GPU-Quicksort for Metal (parallel sorting, Swift library + CLI, Swift 6 / Metal on Apple silicon)

- **Status:** v0.5 — default phase-one pivot changed to `minMaxAverage` and parallel `std::sort` added as a fourth CPU baseline (D-10 revised, D-12 extended, D-23); review findings F-001..F-030 folded in; D-01..D-15 ratified (see §12 and *Revision history*)
- **Language / stack:** Swift 6 (language mode 6) | Metal Shading Language 3.x, Metal framework, swift-argument-parser | C/C++ shim for CPU baselines (libc `qsort`, C++ `std::sort`) | surfaces: Swift library (GPUQuicksort), CLI (`gpuqsort`), build script (`scripts/build-metallib.sh`)
- **Sources:** D. Cederman and P. Tsigas, *GPU-Quicksort: A Practical Quicksort Algorithm for Graphics Processors*, ACM JEA 14, Art. 1.4 (2009), [doi:10.1145/1498698.1564500](https://doi.org/10.1145/1498698.1564500) (the paper is not included in this repository); cited below as [P §n] (sections), [P Alg n] (Algorithms 1–3), [P Fig n], [P Tab n]. Local build host observed while drafting: Apple M5 Max, macOS 26.6, Swift 6.4, Xcode Metal toolchain.
- **Scope of this document:** the sorting algorithm (host orchestration + Metal kernels), its Swift API, the benchmark/verification CLI, the input-distribution generators of [P §5.3], and the tests. Not in scope: the other GPU sorts the paper compares against (GPUSort, radix, hybrid), key–value sorting, stable sorting, non-Apple GPUs.
- **Normative language:** MUST/MUST NOT/SHALL/SHALL NOT = normative; SHOULD = strong recommendation; MAY = optional.
- **Principle:** *Faithful to the paper’s algorithm, correct under Metal’s memory model.* Where the paper’s pseudocode relies on a CUDA behaviour Metal does not guarantee, or contains an off-by-one, this spec keeps the algorithm’s structure (two-pass partition, prefix-sum allocation, atomic block allocation, explicit stack, alternative sort) and changes only the mechanism, and records each change in §12.

0. Intent and purpose

GPU-Quicksort [P §3] is a two-phase parallel Quicksort for GPUs:

- **Phase one** ([P §3.2.1], [P Alg 1–2]): while there are too few independent subsequences for every threadgroup to have its own, many threadgroups cooperate on the same sequence. Each threadgroup counts elements `<` and `>` the pivot in its section (pass 1), computes a prefix sum of those counts, reserves output space for the whole threadgroup with one atomic fetch-and-add per side, and scatters its elements into the auxiliary buffer (pass 2). The host loops until *maxseq* subsequences exist.
- **Phase two** ([P §3.2.2], [P Alg 3]): each threadgroup owns one subsequence and sorts it entirely on the GPU with the same two-pass partition, an explicit stack in threadgroup memory that always processes the smaller part first, and an alternative sort (bitonic) once a part fits in threadgroup memory.

This project re-implements that algorithm for Apple silicon GPUs in Swift + Metal, so the paper’s claims can be revisited on current hardware: that the algorithm is correct for every input distribution, uses $2n + c$ space [P Thm 2], and is bandwidth-bound and faster than a CPU sort [P §5.4].

Mapping of the paper’s CUDA vocabulary used throughout this spec:

Paper (CUDA)	This spec (Metal)
thread block	threadgroup
warp	SIMD-group (width queried at runtime, not assumed)
shared memory	threadgroup memory

Paper (CUDA)	This spec (Metal)
<code>__syncthreads()</code> barrier	<code>threadgroup_barrier(mem_flags::mem_threadgroup)</code>
FAA on global memory	<code>atomic_fetch_add_explicit /</code> <code>atomic_fetch_sub_explicit</code> on device <code>atomic_uint</code>
kernel launch <code>k<<>></code>	compute dispatch of B threadgroups in a <code>MTLComputeCommandEncoder</code>
<code>d, d</code> (primary / auxiliary buffers)	caller buffer D , auxiliary buffer A

Non-goals. Stable sorting; sorting records or key–value pairs (the pivot-fill step in [P §3.1.1 (h)] only works on bare keys, see [I-004](#)); keys wider than 32 bits; guarding against Quicksort’s $O(n^2)$ worst case (no introsort-style fallback); non-Apple GPUs, Intel Macs, iOS; reproducing the paper’s absolute timings; implementing the competitor algorithms.

Relationship to the paper. Each requirement cites the paper passage it comes from. Where this spec differs from the paper, the difference appears in §12 as a **D-nn** row.

1. Actors and goals

Actor	Goals
Library caller (Swift code linking GPUQuicksort)	Sorts a Swift array or a shared <code>MTLBuffer</code> of 32-bit keys in ascending order and gets back a SortReport , or a typed error.
CLI user (<code>gpuqsort</code>)	Generates [P §5.3] distributions, benchmarks GPU-Quicksort against the CPU reference, checks correctness, sorts raw binary files, and prints device limits.
Host orchestrator (Sorter, Swift, CPU)	Runs [P Alg 1]: plans phase-one iterations, encodes dispatches, reads back the partition results, and hands the remaining sequences to phase two. Trusted.
Phase-one kernels (<code>gqsort_partition</code> , <code>gqsort_fill</code> , MSL)	Run [P Alg 2]: several threadgroups cooperate to partition one sequence per iteration. Nothing is ordered across threadgroups except through atomics.
Phase-two kernel (<code>lqsort</code> , MSL)	Runs [P Alg 3]: each threadgroup sorts its own sequence to completion.
CPU reference (<code>CPUReference</code> , Swift)	Sorts the same keys with Swift <code>Array.sort()</code> . It is the correctness oracle.
CPU baselines (<code>CPUBaselines: cpu-swift</code> , <code>cpu-qsort</code> , <code>cpu-stdsort</code> , <code>cpu-stdsort-par</code>)	Performance baselines standing in for the paper’s STL-Introsort [P §5.2]: Swift <code>Array.sort()</code> , libc <code>qsort</code> , C++ <code>std::sort</code> , and C++ <code>std::sort(std::execution::par, ...)</code> through a C-ABI shim (C-11).
Tuner (<code>gpuqsort tune</code>)	Fits the <code>optp</code> constants for the host GPU per [P §5.3] and writes them into the tuned-parameter table (C-10).
Build script (<code>scripts/build-metallib.sh</code>)	Compiles the MSL source into the <code>.metallib</code> resources shipped in the package (C-09).
Metal device (Apple GPU)	Runs the dispatches. It may fail a command buffer, and its limits (threadgroup memory, max threads per threadgroup, max buffer length) are queried, not assumed.

2. Requirements (intent, high level)

2.1 Sorting semantics

ID	Statement
R-01	The library MUST sort n 32-bit keys of type <code>UInt32</code> , <code>Int32</code> or <code>Float</code> into ascending order in the caller’s storage. For <code>Float</code> the order is the IEEE 754-2008 <code>totalOrder</code> (C-04). Source: [P §5.3] (integers and floats).
R-02	After a successful sort, the caller’s storage MUST hold a permutation of the input bit patterns in non-decreasing order under the key’s order (I-001, I-002). For <code>Float</code> , -0.0 sorts before $+0.0$ and NaNs keep their bit patterns.
R-03	The sort MUST be implemented as the two-phase algorithm of [P §3.1]: a host-driven phase one in which several threadgroups share a sequence, then a GPU-resident phase two in which each threadgroup owns one sequence. Phase one MAY be skipped only when E-03 applies, or when $maxseq = 1$ (then the R-08 loop condition is false before the first iteration and phase one performs zero iterations).
R-04	Each partition step, in both phases, MUST use the two-pass scheme of [P §3.1.1]. Pass 1 counts, per thread, the elements $< pivot$ and $> pivot$. A threadgroup-wide prefix sum then gives each thread its write offsets. Pass 2 re-reads the elements and scatters them to the other buffer. Elements equal to the pivot MUST NOT be written by pass 2 (they form the gap, R-06).
R-05	In both passes, thread t of a threadgroup with T threads working on the range $[b, e)$ MUST read indices $b + t, b + t + T, b + 2T + t, \dots$ (“aligned for coalesced reads”, [P Alg 2-3]).
R-06	After partitioning $[s, e)$ with pivot p into L smaller and G greater elements, the gap $[s + L, e - G)$ MUST be filled with p in the caller buffer D . Those indices are final and MUST NOT be part of any later subsequence [P §3.1.1 (h)].
R-07	Every partition MUST read from one buffer and write to the other ($D \leftrightarrow A$). A subsequence MUST record which buffer holds its current elements (C-05, C-06) [P §3.1 “In-place”].

2.2 Phase one

ID	Statement
R-08	Phase one MUST follow [P Alg 1], starting from $work = \{[0, n)\}$ and $done = \emptyset$. Each iteration: compute $blocksize$ (K-06), split every $work$ sequence into $\lceil \ell / blocksize \rceil$ blocks where the last block takes the remainder, dispatch one threadgroup per block, then classify each non-empty child sequence as $done$ if its length is $< minlength$ and as $work$ otherwise. The loop runs while $work \neq \emptyset$ and $ work + done < maxseq$, subject to the iteration cap K-07.
R-09	A phase-one threadgroup MUST reserve output space for the whole threadgroup with exactly one atomic fetch-and-add on the sequence’s low cursor (by L_{tg}) and one atomic fetch-and-subtract on its high cursor (by G_{tg}), issued by one thread, and MUST then share the result with the other threads through threadgroup memory and a barrier [P §3.2.1 “Space Allocation”, Alg 2].
R-10	The pivot fill of R-06 and the derivation of child sequences and their pivots MUST happen only after every threadgroup of that iteration’s partition dispatch has completed. This is done with a follow-up <code>gqsort_fill</code> dispatch in the same command buffer plus a host read-back after the command buffer completes. It replaces the paper’s “last block to finish” step (D-03).

ID	Statement
R-11	Phase-one pivots MUST be chosen by the configured <code>PhaseOnePivot</code> strategy (C-03). The default is <code>minMaxAverage</code> ([P §5.2]; specified by O-2; D-10 revised in v0.5). <code>medianOfThree</code> MAY be selected: the median of s_b , $s_{\lfloor(b+e)/2\rfloor}$ and s_{e-1} of the sequence $[b, e)$ in its current buffer ([P Alg 1–2]).

2.3 Phase two

ID	Statement
R-12	Phase two MUST dispatch one threadgroup per sequence in <code>done</code> (after <code>work</code> is merged into <code>done</code> , [P Alg 1]). Each threadgroup MUST sort its sequence to completion without communicating with other threadgroups [P §3.2.2].
R-13	A phase-two threadgroup MUST keep pending subsequences on an explicit stack in threadgroup memory (capacity K-08). After each partition it MUST push the longer child and then the shorter child, skipping empty children and children handled by R-15, and MUST pop from the top, so the shorter child is always processed first [P §3.2.2 “Stack”, Alg 3].
R-14	Phase-two pivots MUST be the median of s_b , $s_{\lfloor(b+e)/2\rfloor}$ and s_{e-1} of the current range $[b, e)$ [P Alg 3, §5.2].
R-15	A sequence whose length is $< minseq$ (the paper’s MINSIZE / sbsize) MUST be sorted by the alternative sort. The alternative sort loads the sequence into threadgroup memory, pads it to the next power of two with <code>0xFFFFFFFF</code> , bitonic-sorts it, and writes the first ℓ elements to D at their final positions [P §3.1 “Second Phase”, §3.2.2 “Overhead”].

2.4 Parameters, API, CLI, and cross-cutting

ID	Statement
R-16	The sort MUST take three tuning parameters [P §5.3]: threads per threadgroup T , maximum phase-one sequences $maxseq$, and minimum Quicksort sequence length $minseq$. Each parameter the caller omits MUST default to $optp(n, k, m)$ (K-05) with the constants from the tuned-parameter table (C-10), clamped per K-04.
R-17	<code>Int32</code> and <code>Float</code> keys MUST be converted to order-preserving <code>UInt32</code> codes (C-04) before phase one and converted back after phase two, on the GPU, in the caller’s buffer. The algorithm itself sorts only unsigned 32-bit codes.
R-18	The library MUST expose the API of C-01/C-02. Every failure MUST be reported as a typed <code>GPUQuicksortError</code> (C-07). The library MUST NOT crash, trap or log key values on any input.
R-19	The CLI MUST provide the subcommands <code>bench</code> , <code>verify</code> , <code>sort</code> , <code>gen</code> , <code>info</code> and <code>tune</code> (§5.2), with the exit codes of K-12.
R-20	The generators MUST produce the six distributions of [P §5.3] (uniform, sorted, zero, bucket, gaussian, staggered) exactly as C-08 defines them, from MT19937 seeded by the caller, so a given (distribution, n , seed) always yields the same bytes.
R-21	Every sort MUST return a <code>SortReport</code> (C-02) with wall-clock time, GPU time, the parameters actually used, phase-one iteration and sequence counts, phase-two partition and alternative-sort counts, and the maximum stack depth any phase-two threadgroup reached.

ID	Statement
R-22	Diagnostics: for every sort the library MUST emit one line per phase-one iteration and one summary line, in the exact formats of §5.3. Each line MUST go to <code>os.Logger</code> (subsystem <code>GPUQuicksort</code> , <code>.debug</code> level) and, when set, to the <code>diagnostics</code> handler of C-01, with identical text. The CLI MUST be silent on <code>stderr</code> unless <code>--verbose</code> is given, in which case it installs a handler that writes each line to <code>stderr</code> . Key values MUST NOT appear in any line.
R-23	<code>bench</code> MUST time only the sort (C-02 <code>wallTime</code>). Time spent generating data, allocating buffers, creating the sorter (<code>init</code>), restoring the unsorted input before each run, and verifying MUST be excluded, and the first (warm-up) run of each configuration MUST be discarded [P §5.2 “We only measured the actual sorting phase”].
R-24	<code>gpuqsort tune</code> MUST reproduce the procedure of [P §5.3, Fig 9–11, Tab II] on the host GPU. For each size in the list it grid-searches $(T, maxseq, minseq)$, keeps the configuration with the lowest median <code>wall_ms</code> , then fits (k, m) per parameter by least squares (C-10), and prints the result. With <code>--write</code> , it stores the fit in the tuned-parameter table under the device’s name (D-06). <code>tune</code> MUST create its sorter with <code>TuningSource.constants(.paper8800GTX)</code> and pass every parameter explicitly, so it never depends on the table it writes.
R-25	The shipped package MUST contain a valid tuned-parameter table (C-10) at every commit. Before the first tuning run it holds only the bootstrap entry <code>paper-8800gtx</code> (the [P Tab II] 8800GTX constants), with <code>apple-default</code> pointing at it. The conforming release MUST additionally contain an entry for the reference machine (Apple M5 Max) produced by R-24, with <code>apple-default</code> pointing at that entry (D-16).
R-26	<code>bench --cpu</code> MUST time four CPU baselines on the same input: <code>cpu-swift</code> (Swift <code>Array.sort()</code>), <code>cpu-qsort</code> (libc <code>qsort</code>), <code>cpu-stdsort</code> (C++ <code>std::sort</code>), and <code>cpu-stdsort-par</code> (C++ <code>std::sort(std::execution::par, ...)</code> , libc++’s parallel algorithms). Each sorts the C-04 codes as <code>UInt32</code> (C-11). Correctness is always judged against <code>CPUReference</code> (Swift <code>Array.sort()</code>) (D-12, D-23).
R-27	The Metal code MUST be delivered as <code>.metallib</code> files precompiled by <code>scripts/build-metallib.sh</code> and loaded at <code>init</code> with <code>MTLDevice.makeLibrary(URL:)</code> . The library MUST NOT compile MSL source at runtime. The package MUST fail its test suite if a shipped <code>.metallib</code> is stale relative to its source (C-09, D-05).
R-28	Ordering inside one threadgroup (<code>lqsort</code> ; rule (a) also for <code>gqsort_partition</code>), because Metal device memory is not coherent between threads without a barrier: (a) the gap fill of a partition MUST NOT begin until every thread has finished pass 2 of that partition; (b) before any thread reads elements that other threads of the same threadgroup wrote to device memory (the next popped sequence, or an alternative-sort load), the threadgroup MUST execute <code>threadgroup_barrier(mem_flags::mem_device mem_flags::mem_threadgroup)</code> ; (c) the alternative sort MUST finish loading its sequence into threadgroup memory, followed by a threadgroup barrier, before any thread writes the result back to <i>D</i> . The same rule (a) applies to <code>gqsort_partition</code> pass 2 versus the counters of pass 1.

2.5 Optional items

ID	Statement
Ø-1	Retired in v0.2: <code>tune</code> is required (R-24).

ID	Statement
O-2	minMaxAverage phase-one pivot (the default since v0.5, D-10; <code>Parameters.phaseOnePivot = .minMaxAverage</code> or <code>--pivot minmax</code>; the alternative <code>medianOfThree</code> is selected with <code>--pivot median</code>). The root sequence $[0, n)$ uses the <code>medianOfThree</code> pivot (D-20). For every other phase-one sequence with minimum code lo and maximum code hi, the pivot MUST be $p = lo + \lfloor (hi - lo)/2 \rfloor$, computed in unsigned 32-bit arithmetic (no overflow). During pass 2 each <code>gqsort_partition</code> threadgroup MUST reduce the min and max of the elements it writes to each side in threadgroup memory and then apply exactly one <code>atomic_fetch_min_explicit/atomic_fetch_max_explicit</code> per field to <code>lmin</code>, <code>lmax</code> (the $<$ side) and <code>gmin</code>, <code>gmax</code> (the $>$ side) of C-05. After read-back, the $<$ child's pivot uses $(lmin, lmax)$ and the $>$ child's uses $(gmin, gmax)$. Because $lo \leq p < hi$ whenever $lo < hi$, and $p = lo = hi$ otherwise, every child is strictly shorter than its parent (I-005). Under <code>medianOfThree</code> the kernel MUST NOT touch the four fields.

3. Behavior and state model

3.1 Lifecycle of one sort call

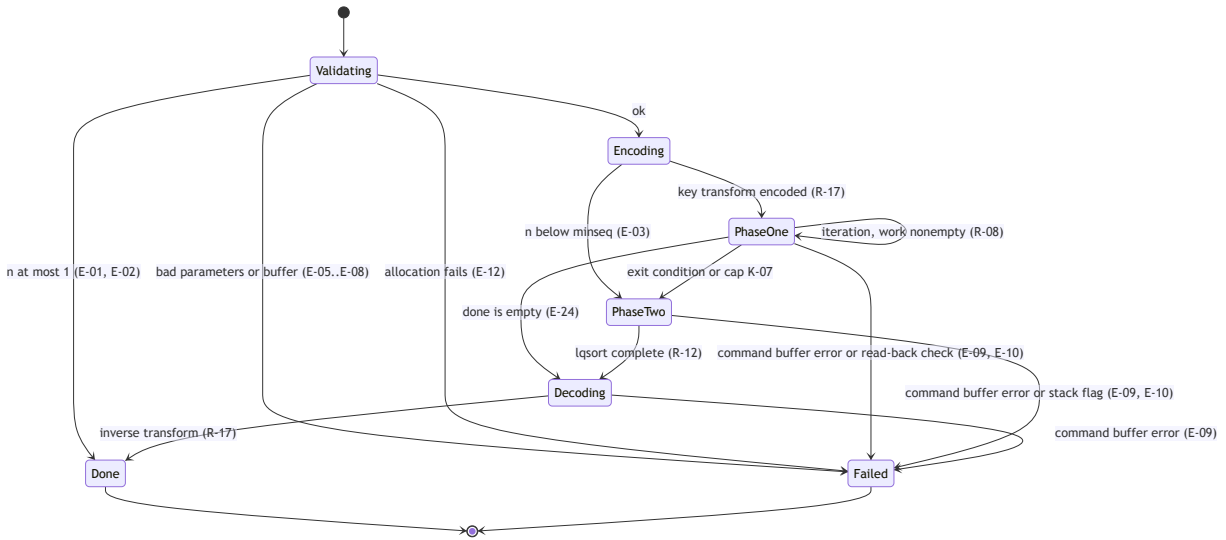


Figure 3.1 — sort-call lifecycle per [R-08](#), [R-12](#), [R-17](#), [K-07](#), [E-01..E-03](#), [E-05..E-10](#), [E-12](#), [E-24](#). The transitions table below is normative.

From	Trigger	To	Observable result
Validating	parameters or buffer invalid	Failed	throws C-07 error; caller buffer untouched (I-006)
Validating	$n \leq 1$	Done	no GPU work; report with zero counters
Validating	valid, $n \geq 2$	Encoding	aux buffer A obtained (K-09)
Validating	allocation of A or a descriptor buffer fails	Failed	throws <code>allocationFailed</code> ; caller buffer untouched (E-12 , I-006)
Encoding	$n < minseq$	PhaseTwo	<code>done</code> = $\{[0, n)\}$ in D ; phase-one iteration count 0
Encoding	otherwise	PhaseOne	—

From	Trigger	To	Observable result
PhaseOne	iteration finished, loop condition of R-08 holds, iterations < K-07	PhaseOne—	
PhaseOne	loop condition false (including zero iterations when $maxseq = 1$, F-005), or K-07 reached, and <code>done</code> $\neq \emptyset$ after merging	PhaseTwo	<code>work</code> merged into <code>done</code>
PhaseOne	as above, but <code>done</code> = $\emptyset$ after merging	Decoding	no <code>lqsort</code> dispatch (E-24)
PhaseOne	a read-back check fails (E-10)	Failed	throws <code>internalInvariantViolated</code>
PhaseTwo	<code>lqsort</code> command buffer completes and no error flag is set	Decoding—	
any GPU state	command buffer <code>.error</code> , or kernel error flag	Failed	throws; caller buffer contents unspecified (E-09)
Decoding	complete	Done	report returned

3.2 One phase-one iteration

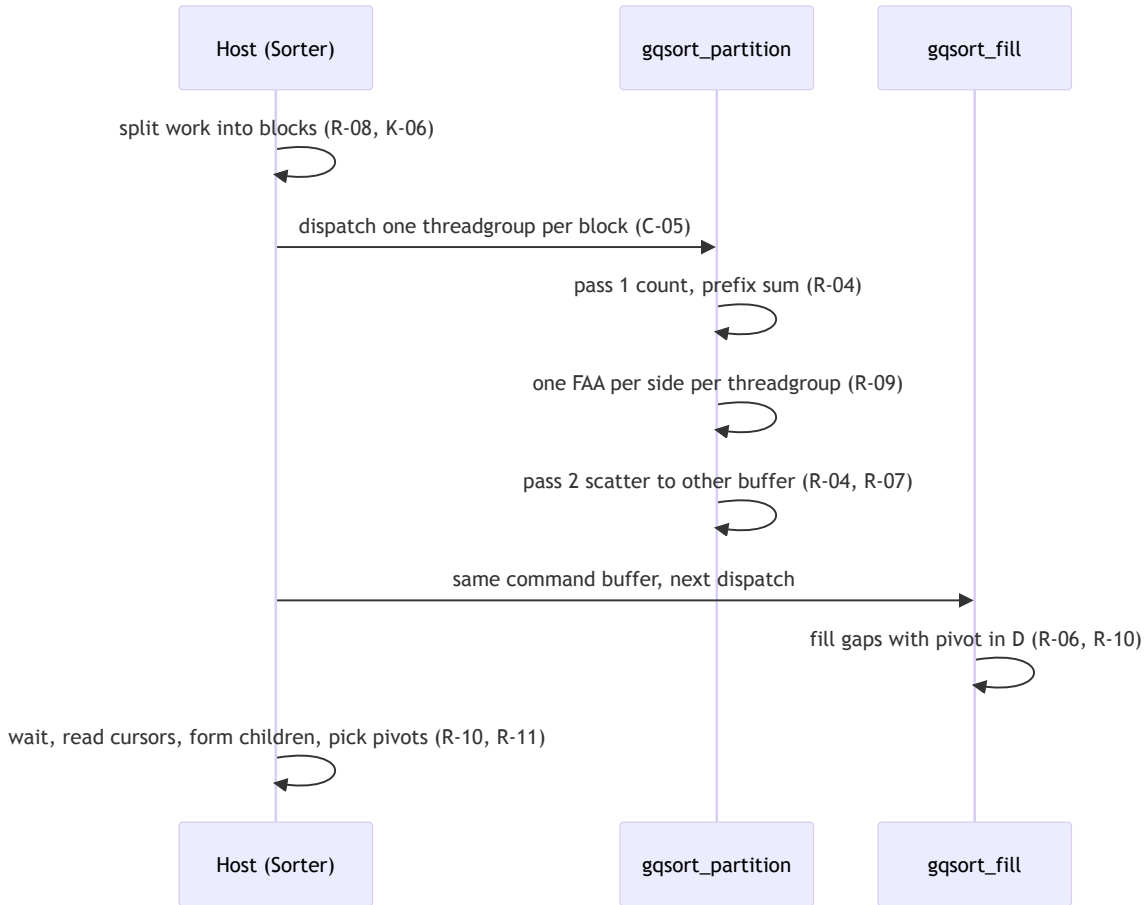


Figure 3.2 — one phase-one iteration per [R-04](#), [R-06..R-11](#), [K-06](#).

3.3 Buffers

Two buffers of n codes each exist during a sort: the caller's D and the auxiliary A ([K-09](#)). A sequence's elements live in exactly one of them (its `src`). Partitioning moves them to the other. Final values (pivot gaps, alternative-sort output) always go to D . Descriptor and statistics buffers are $O(maxseq)$ in size. There are no durable artifacts. The CLI's files are described in §5.2.

4. Interfaces / contracts

C-01 Public API

```

import Metal

public final class GPUQuicksort: @unchecked Sendable {
    public static let version: String // semantic version of the package, e.g. "0.3.0"

    /// Loads the precompiled GPUQuicksort.metallib (C-09), builds the pipelines for `device`
    /// (default: MTLCreateSystemDefaultDevice()), and resolves tuning constants from `tuning` (C-10).
    /// Throws .noMetalDevice, .unsupportedDevice, .shaderLibraryMissing, .shaderLibraryLoadFailed,
    /// .tunedParametersInvalid (only for .bundled / .file).
    public init(device: MTLDevice? = nil, tuning: TuningSource = .bundled) throws
  
```

```

public var device: MTLDevice { get }
public var limits: DeviceLimits { get }           // C-03
public var tuning: TunedConstants { get }         // C-10: the constants in effect
public var metallibSHA256: String { get }         // contents of Resources/metallib.sha256 (C-09)

/// Receives every diagnostic line of R-22 / §5.3, synchronously, on the thread that called sort.
/// Getter and setter take the lock that serializes sort calls; a set during a running sort
/// blocks until that sort returns and takes effect for the next sort.
public var diagnostics: (@Sendable (String) -> Void)?

/// Sorts `count` keys starting at byte 0 of `buffer`. buffer.storageMode MUST be .shared.
/// Synchronous: blocks the calling thread until the GPU work completes; not cancellable.
@discardableResult
public func sort(_ buffer: MTLBuffer, count: Int, keyType: KeyType,
                parameters: Parameters = .automatic) throws -> SortReport

/// Convenience: copies into a shared buffer, sorts, and copies back.
/// Copy time is included in wallTime.
@discardableResult
public func sort<K: GPUSortableKey>(_ keys: inout [K],
                                   parameters: Parameters = .automatic) throws -> SortReport

/// Resolves .automatic/partial parameters for n exactly as sort() would (K-04, K-05).
public func resolvedParameters(for n: Int, _ p: Parameters) throws -> ResolvedParameters
}

public enum KeyType: String, Sendable, CaseIterable { case uint32, int32, float32 }

public enum TuningSource: Sendable {
    case bundled           // Resources/TunedParameters.json, lookup per C-10
    case file(URL)         // a table with the C-10 schema, lookup per C-10
    case constants(TunedConstants) // no table; used as given (still validated: k 0, m 1)
}

public struct TunedConstants: Sendable, Codable, Equatable {
    public struct Line: Sendable, Codable, Equatable { public var k: Double; public var m: Double }
    public var entry: String           // table key it came from, or "constants"
    public var exactMatch: Bool        // true iff the entry key equals MTLDevice.name
    public var threads: Line, maxseq: Line, minseq: Line
    public static let paper8800GTX: TunedConstants // [P Tab II] 8800GTX row; entry "paper-8800gtx"
}

public protocol GPUSortableKey: BitwiseCopyable { static var keyType: KeyType { get } }
extension UInt32: GPUSortableKey {} // .uint32
extension Int32: GPUSortableKey {} // .int32
extension Float: GPUSortableKey {} // .float32

```

A GPUQuicksort instance MUST serialize concurrent `sort` calls internally, so that calls from different threads run one after another (D-09). Parameter validation and resolution happen before the caller buffer is touched.

C-02 SortReport

```

public struct SortReport: Sendable, Codable, Equatable {
    public var count: Int
    public var keyType: KeyType
    public var parameters: ResolvedParameters
}

```

```

    public var wallTime: Double           // seconds, host monotonic clock (K-11)
    public var gpuTime: Double            // seconds,  $\Sigma$  (gpuEndTime - gpuStartTime) over all command
    public var phaseOneIterations: Int    // 0 when phase one is skipped
    public var phaseOneSequences: Int     // |done| handed to phase two
    public var phaseOneCapReached: Bool   // K-07: iterations == max AND the R-08 loop condition sti
    public var phaseTwoPartitions: Int    //  $\Sigma$  partitions over all lqsort threadgroups
    public var phaseTwoAltSorts: Int      //  $\Sigma$  alternative sorts (R-15)
    public var maxStackDepth: Int         // max over threadgroups of the stack high-water mark:
                                         // entries on the stack immediately after any push,
                                         // the initial push counting as 1 (K-08)

    public var auxiliaryBytes: Int        // bytes of the auxiliary buffer A (K-09)
    public var bookkeepingBytes: Int      // bytes of all other per-sort device allocations (K-09)
    public var libraryVersion: String     // GPUQuicksort.version
    public var metallibSHA256: String     // provenance (F-014)
    public var tuningEntry: String        // TunedConstants.entry in effect
}

```

C-03 Parameters and device limits

```

public struct Parameters: Sendable, Codable, Equatable {
    public var threadsPerThreadgroup: Int? // T; nil → optp default (K-05)
    public var maxSequences: Int?          // maxseq; nil → optp default
    public var minSequenceLength: Int?     // minseq; nil → optp default
    public var phaseOnePivot: PhaseOnePivot = .minMaxAverage // D-10 (revised v0.5)
    public var maxPhaseOneIterations: Int = 64 // K-07
    public static let automatic = Parameters()
}

public enum PhaseOnePivot: String, Sendable, Codable { case medianOfThree, minMaxAverage } // 0-2
public struct ResolvedParameters: Sendable, Codable, Equatable {
    public var threadsPerThreadgroup: Int, maxSequences: Int, minSequenceLength: Int
    public var phaseOnePivot: PhaseOnePivot, maxPhaseOneIterations: Int
}

public struct DeviceLimits: Sendable, Codable, Equatable {
    public var name: String
    public var maxThreadsPerThreadgroup: Int // min over the three pipelines' maxTotalThreadsPerThr
    public var threadExecutionWidth: Int
    public var maxThreadgroupMemoryLength: Int
    public var maxBufferLength: Int
    public var maxKeys: Int // K-01
}

```

C-04 Key encoding (order-preserving map to UInt32)

Let b be the 32-bit pattern of a key and u its code. The following are applied element-wise on the GPU (key_encode, key_decode); uint32 is the identity and needs no dispatch.

```

int32  encode: u = b XOR 0x8000_0000          decode: b = u XOR 0x8000_0000
float32 encode: u = (b & 0x8000_0000) ? ~b : b XOR 0x8000_0000
        decode: b = (u & 0x8000_0000) ? u XOR 0x8000_0000 : ~u

```

For float32, unsigned order on u equals IEEE 754-2008 totalOrder: $-\text{NaN} < -\infty < \dots < -0 < +0 < \dots < +\infty < +\text{NaN}$, with NaNs ordered by payload. Encode and decode MUST be exact inverses on all 2^{32} patterns (T-05).

C-05 Phase-one descriptors (host gqsort_partition / gqsort_fill)

These layouts are shared between Swift and MSL and MUST be declared once, in `Sources/CShared/include/SharedTypes.h`, which both the Metal compiler and Swift (through the `CShared` target) include. Swift does not import C11 `_Atomic` fields, so the header MUST declare atomic fields through a macro: `atomic_uint` in MSL, plain `uint32_t` elsewhere. The host writes and reads those fields only while no GPU work that uses the buffer is in flight. All fields are little-endian 32-bit. [T-31](#) checks that both sides agree.

```
#ifdef __METAL_VERSION__
    #include <metal_stdlib>
    #define GQS_ATOMIC_U32 metal::atomic_uint
#else
    #include <stdint.h>
    #define GQS_ATOMIC_U32 uint32_t
#endif

typedef struct {
    uint32_t start;           // one per sequence in the current iteration's work set
    uint32_t end;            // oldstart: first index of the sequence
    GQS_ATOMIC_U32 lnext;    // oldend: one past the last index
    GQS_ATOMIC_U32 gnext;    // sstart: next free index for "< pivot"; host initializes to start
    uint32_t pivot;         // send: one past the last free index for "> pivot"; host initializes to end
    uint32_t src;           // encoded pivot code
    GQS_ATOMIC_U32 lmin, lmax, gmin, gmax; // 0 = elements currently in D, 1 = in A; writes go to the other
    // 0xFFFFFFFF, 0, 0xFFFFFFFF, 0; untouched under medianOfThree
} SequenceRecord;          // 40 bytes

typedef struct {
    uint32_t begin, end;    // one per threadgroup of a gqsort_partition dispatch
    uint32_t seq;           // [begin, end) section of the parent sequence (K-06)
    uint32_t _pad;         // index into the SequenceRecord array
} BlockDescriptor;        // 16 bytes
```

After the command buffer completes, sequence j 's children are $[start, lnext)$ and $[gnext, end)$, both in buffer $1 - src$. Its gap is $[lnext, gnext)$, already filled in D by `gqsort_fill`.

C-06 Phase-two descriptors (host lqsort)

```
typedef struct { uint32_t begin, end, src, _pad; } SortSequence; // one per threadgroup
typedef struct {
    uint32_t partitions; // one per threadgroup, written by thread 0 on exit
    uint32_t altSorts;    // partition steps performed
    uint32_t altSorts;    // alternative sorts performed
    uint32_t maxDepth;    // stack high-water mark
    uint32_t error;       // stack high-water mark
    // 0 = ok; 1 = stack overflow (E-10)
} SortStats;
```

The stack entry is $\{begin, end, src\}$ ($3 \times \text{uint32}$) in threadgroup memory.

C-07 Errors

```
public enum GPUQuicksortError: Error, Equatable, Sendable {
    case noMetalDevice
    case unsupportedDevice(String) // lacks Apple7 GPU family (K-02)
    case shaderLibraryMissing(String) // resource path of the missing .metallib
    case shaderLibraryLoadFailed(String) // makeLibrary(URL:) / makeFunction / pipeline error text
    case tunedParametersInvalid(String) // C-10 resource missing or fails validation
    case invalidParameters(String) // K-04 violated by an explicit value; names the parameter
```

```

    case bufferNotShared // E-06
    case bufferTooSmall(required: Int, actual: Int) // E-07
    case tooManyKeys(count: Int, max: Int) // E-08, K-01
    case allocationFailed(bytes: Int) // E-12
    case gpuExecutionFailed(String) // E-09: MTLCommandBuffer.error description
    case internalInvariantViolated(String) // E-10 and any C-05/C-06 sanity check on read-back
}

```

C-08 Distributions ([P §5.3])

All draws come from one MT19937 (32-bit) engine, seeded with `seed: UInt32` (default 42), and are consumed in element-index order. Let n be the length, $k \in [0, n)$ the element index, $p = 128$, $w = 2^{31}/p = 2^{24}$, and r the next 32-bit draw. The notation $U(a, len) = a + (r \bmod len)$ means “one draw, offset into a range of length len ” (every len used here is a power of two, so there is no modulo bias).

Name	Value v_k (a UInt32 in $[0, 2^{31})$)
uniform	$U(0, 2^{31})$
sorted	the uniform sequence, sorted ascending on the CPU
zero	one draw $c = U(0, 2^{31})$ at the start; $v_k = c$ for all k
bucket	section $S = \lfloor kp^2/n \rfloor \bmod p$ (the section of element k within its block $\lfloor kp/n \rfloor$); $v_k = U(Sw, w)$
gaussian	$v_k = \lfloor (u_1 + u_2 + u_3 + u_4)/4 \rfloor$, where $u_1, \dots, u_4 = U(0, 2^{31})$ are four consecutive draws (so element k consumes draws $4k..4k+3$) and the sum is computed in 64-bit unsigned arithmetic
staggered	block $i = \lfloor kp/n \rfloor$ (0-based); if $i < p/2$: $U((2i+1)w, w)$, else $U((2i-p)w, w)$ (D-07)

Products such as kp^2 and the **gaussian** sum MUST use 64-bit arithmetic. For **int32**, v_k is reinterpreted as **Int32** (all values non-negative). For **float32**, v_k becomes **Float(v_k)** (round-to-nearest). A separate **fullrange** distribution, $v_k = r$ reinterpreted as the key’s bit pattern, MUST exist for tests; it covers negative integers and every float class including NaN.

C-09 Package layout and shader delivery

```

Package.swift // swift-tools-version 6.0; platforms: .macOS(.v15)
scripts/build-metallib.sh // MSL → .metallib (below); run after any .metal/.h change
Sources/GPUQuicksort/ // library target (C-01..C-08, C-10); Package.swift: exclude: ["Meta
Sources/GPUQuicksort/Metal/GPUQuicksort.metal // source of truth for kernels (excluded, not a res
Sources/GPUQuicksort/Resources/GPUQuicksort.metallib // resources: [.copy("Resources")]
Sources/GPUQuicksort/Resources/GPUQuicksort-testhooks.metallib // built with -DGPUQS_TEST_HOOKS
Sources/GPUQuicksort/Resources/metallib.sha256 // stamp: SHA-256 of the inputs (below)
Sources/GPUQuicksort/Resources/TunedParameters.json // C-10
Sources/CShared/include/SharedTypes.h // C-05/C-06 layouts; public header of the C target
Sources/CShared/CShared.c // empty translation unit (SwiftPM needs one source
Sources/CPUBaselines/ // C/C++ target: qsort + std::sort shim (C-11)
Sources/gpuqsort/ // executable target, depends on swift-argument-parser
Tests/GPUQuicksortTests/ // swift-testing (T-xx)

```

`scripts/build-metallib.sh` writes to `$OUT_DIR` (environment variable; default `Sources/GPUQuicksort/Resources`). It MUST, for each of the two variants (release, and `-DGPUQS_TEST_HOOKS`):

```

xcrun -sdk macosx metal -std=metal3.1 -mmacosx-version-min=15.0 -O3 \
    [-DGPUQS_TEST_HOOKS] -I Sources/CShared/include \
    -c Sources/GPUQuicksort/Metal/GPUQuicksort.metal -o "$TMP/<variant>.air"
xcrun -sdk macosx metallib "$TMP/<variant>.air" -o "$OUT_DIR/<variant>.metallib"

```

It then writes `$OUT_DIR/metallib.sha256`: one line, the lowercase hex SHA-256 of the concatenated bytes of `Sources/GPUQuicksort/Metal/GPUQuicksort.metal` followed by `Sources/CShared/include/SharedTypes.h`.

The script MUST build into a temporary directory and move the results into `$OUT_DIR` only after every step succeeds, so that on any failure it exits non-zero and leaves the previous files in place. The `.metallib` files and the stamp are checked in, so `swift build` needs no Metal compiler (D-05).

`init` MUST load `GPUQuicksort.metallib` from `Bundle.module` with `makeLibrary(URL:)`. Test hooks are enabled by `swiftSettings: [.define("GPUQS_TEST_HOOKS", .when(configuration: .debug))]` on the library and CLI targets (D-21). A build with that define loads the `-testhooks` variant instead, and a release build never contains hooks. Kernel function names: `key_encode`, `key_decode`, `gqsort_partition`, `gqsort_fill`, `lqsort`. Everything else is internal.

C-10 Tuned-parameter table

```
{
  "schema": 1,
  "entries": {
    "paper-8800gtx": {
      "fitted": "2009-07-01", "gpuqsortVersion": "paper", "sizes": [],
      "threads": {"k": 0.00001172, "m": 53},
      "maxseq": {"k": 0.00003748, "m": 476},
      "minseq": {"k": 0.00004685, "m": 211}
    },
    "apple-default": { "sameAs": "paper-8800gtx" }
  }
}
```

This is the bootstrap table that ships until the first tuning run (R-25). After `gpuqsort tune --write --as-default` on the reference machine, the table gains an "Apple M5 Max" entry of the same shape (with `fitted`, `gpuqsortVersion` and `sizes` filled in), and `apple-default` becomes `{"sameAs": "Apple M5 Max"}`. The `paper-8800gtx` entry is kept.

- **Lookup** (for `.bundled` and `.file`): use the exact `MTLDevice.name`; otherwise `apple-default` (following one `sameAs` hop). The result is exposed as `GPUQuicksort.tuning` (C-01), where `entry` is the key finally used (after the hop) and `exactMatch` tells whether the device name matched. `.constants` bypasses the table.
- **Validation at init**: for a table, `schema == 1`, `apple-default` is present, `sameAs` targets exist and are not themselves `sameAs`; for every entry and for `.constants`, every $k \geq 0$ and $m \geq 1$, and all values are finite. Otherwise `init` throws `tunedParametersInvalid`.
- **Fit (R-24)**: for parameter x , with measured best values x_j at sizes s_j ($j = 1..J$, $J \geq 2$), (k, m) minimize $\sum_j (ks_j + m - x_j)^2$ (ordinary least squares, as in [P §5.3]). If $k < 0$, set $k = 0$ and $m = \bar{x}$. Then set $m = \max(m, 1)$. With $J = 1$: $k = 0$, $m = x_1$.
- **--write**: the table file is validated *before* the grid search starts. (1) If the file does not exist, `tune` creates a table holding only the new entry, with `apple-default` pointing at it whether or not `--as-default` is given. (2) If the file exists and is a valid C-10 table, `tune` replaces or inserts the entry for `MTLDevice.name`, keeps the other entries, and with `--as-default` also sets `apple-default` to `{"sameAs": <name>}`. (3) If the file exists but is not a valid C-10 table, `tune` exits 4 with `gpuqsort: error: table <path> is not a valid tuned-parameter table: <reason>` before measuring anything, and leaves the file unchanged. The written table MUST itself pass C-10 validation; it is written with sorted keys and 2-space indentation, via a temporary file and an atomic rename (E-23). Without `--write`, the table file is never read. The file path defaults to the package resource path in the source tree (`--table <path>` overrides it).

C-11 CPU baselines (C/C++ shim)

```
// Sources/CPUBaselines/include/CPUBaselines.h - C ABI, callable from Swift
#include <stddef.h>
#include <stdint.h>
```

```

void cpub_qsort_u32(uint32_t *keys, size_t n);    // libc qsort with a (a>b)-(a<b) comparator
void cpub_stdsort_u32(uint32_t *keys, size_t n); // std::sort(keys, keys+n), compiled -O3, C++17
void cpub_stdsort_par_u32(uint32_t *keys, size_t n); // std::sort(std::execution::par, ...), -O3, C++17,
                                                    // -fexperimental-library (libc++ parallel algorithm)

```

cpu-swift is `[UInt32].sort()` on the codes. For `int32/float32`, every baseline’s timed region includes the CPU encode and decode of [C-04](#) (the GPU’s timing includes its encode/decode too), so the comparison is end-to-end on the same input and output. The baseline implementations MUST be built in release configuration when benchmarked. `bench` and `tune` refuse to run from a debug build unless `--allow-debug` is passed (exit 2), and recorded runs ([T-32](#), [T-34](#)) MUST use a release build.

5. Interface specification

5.1 Library

Operation	Inputs	Output	Errors
<code>init(device:tuning:)</code>	optional device, <code>TuningSource</code> (default <code>.bundled</code>)	ready sorter; pipelines built from the metallib; tuning resolved	<code>noMetalDevice</code> , <code>unsupportedDevice</code> , <code>shaderLibraryMissing</code> , <code>shaderLibraryLoadFailed</code> , <code>tunedParametersInvalid</code> C-07 per §8
<code>sort(_:count:keyType:parameters:)</code>	shared buffer, $0 \leq$ count, key type, parameters	<code>SortReport</code> ; buffer sorted in place	
<code>sort(_:parameters:)</code>	inout [K]	<code>SortReport</code> ; array sorted	as above except <code>bufferNotShared</code> / <code>bufferTooSmall</code>
<code>resolvedParameters(for:_:)</code>	n , parameters	<code>ResolvedParameters</code>	<code>invalidParameters</code> , <code>tooManyKeys</code>
<code>limits</code>	—	<code>DeviceLimits</code>	—

5.2 CLI `gpuqsort`

Global flags: `--verbose` ([R-22](#)), `--table <path>` (use `TuningSource.file(path)` instead of the bundled table, for `info`, `sort`, `verify`, `bench`), `--help`, `--version` (prints `GPUQuicksort.version`). Size lists accept suffixes $K = 2^{10}$ and $M = 2^{20}$ (e.g. `1M`, `2M`, `16M`).

Subcommand	Arguments (defaults)	Behaviour	Output
info	<code>--json</code>	prints the library version, <code>DeviceLimits</code> , the tuning constants in effect (entry and exact match), the metallib stamp, and the resolved defaults for $n \in \{2^{20}, 2^{24}\}$	human text on stdout; with <code>--json</code> , one object <code>{version, metallibSHA256, limits: DeviceLimits, tuning: TunedConstants, defaults: [{n, parameters: ResolvedParameters}]}</code> carrying the same information
gen	<code>--dist</code> (required), <code>--n</code> (required), <code>--key uint32</code> , <code>--seed 42</code> , <code>--out</code> (required)	writes C-08 values	raw little-endian 4-byte keys, exactly $4n$ bytes
sort	<code>--in</code> (required), <code>--out</code> (required), <code>--key uint32</code> , tuning flags	reads, sorts, writes	output file as <code>gen</code> ; report on stderr only if <code>--verbose</code>
verify	<code>--dist all</code> , <code>--n 1K,1M</code> , <code>--key all</code> , <code>--seed 42</code> , <code>--runs 1</code> , tuning flags	sorts each (dist, n, key) and compares bit-for-bit against <code>CPUReference</code> (T-01)	one line per case <code>PASS/FAIL dist n key</code> on stdout; exit 1 if any <code>FAIL</code>
bench	<code>--dist all</code> , <code>--n 1M,2M,4M,8M,16M</code> , <code>--key uint32</code> , <code>--runs 5</code> , <code>--seed 42</code> , <code>--cpu</code> (include the three CPU baselines, R-26), <code>--allow-debug</code> , <code>--format csv</code> , tuning flags	R-23 timing; the oracle output is computed once per (key, dist, n , seed) with <code>CPUReference</code> , and each timed run is verified by a byte comparison against it; a mismatch aborts with exit 1 after flushing the rows already produced	CSV or JSON (below) on stdout

Subcommand	Arguments (defaults)	Behaviour	Output
tune	<code>--n 512K,1M,2M,4M,8M,16M, --dist uniform, --key uint32, --runs 3, --seed 42, --write, --as-default, --table <path>, --allow-debug</code>	grid search over every valid combination (K-04 , K-03) of $T \in \{32, 64, 128, 256, 512, 1024\}$, $maxseq \in \{32, 64, \dots, 4096\}$ and $minseq \in \{64, 128, \dots\}$; per size, one warm-up and <code>--runs</code> timed runs per configuration, each verified by a byte comparison against an oracle computed once per size; best = lowest median <code>wall_ms</code> , ties broken by smaller T , then smaller $maxseq$, then smaller $minseq$; fit per C-10 ; progress is printed to stderr only with <code>--verbose</code>	JSON on stdout: <code>{device, gpusort_version, meta, allib_sha256, os_version, sizes, best: [{n, threads, maxseq, m}, ...], fit: {threads: {k, m}, maxseq: {k, m}}, grid: [{n, threads, maxseq, m}, ...]}</code> with <code>--write</code> , also updates C-10

Tuning flags, shared by `sort`, `verify`, `bench`: `--threads T`, `--maxseq N`, `--minseq N`, `--pivot minmax|median` (default `minmax`, [D-10](#)).

`bench` CSV (RFC 4180: a field containing a comma, double quote or newline is enclosed in double quotes, with inner double quotes doubled). Header (exact, one row per timed run):

`device,key,distribution,n,run,algorithm,wall_ms,gpu_ms,threads,maxseq,minseq,phase1_iterations,phase1_size`

`algorithm` is one of `gpu-quicksort`, `cpu-swift`, `cpu-qsort`, `cpu-stdsort`, `cpu-stdsort-par`. For the `cpu-*` rows, `gpu_ms`, the tuning columns and `tuning_entry` are empty. `os_version` is `ProcessInfo.operatingSystemVersionString`. `--format json` emits an array of objects with the same keys. After the rows, and only with `--verbose`, a summary per (dist, n, algorithm) goes to stderr: the median and min of `wall_ms` and the throughput $n/(\text{median } wall_ms \cdot 10^3)$ in Mkeys/s ([K-11](#)).

5.3 Cross-cutting interface contracts

- **Errors** → **exit codes** ([K-12](#)). CLI error messages MUST go to stderr as `gpusort: error: <message>`, where `<message>` is the [C-07](#) case and its detail.
- **Diagnostics** ([R-22](#)). At the default verbosity the CLI writes nothing to stderr on success. The library emits, and with `--verbose` the CLI prints, exactly these lines (fields separated by one space, numbers in decimal, times in ms with 3 decimals):
 - after each phase-one iteration: `phase1 iter=<i> work=<w> done=<d> threadgroups=<b> ms=<t>`, where i counts from 1, w and d are $|work|$ and $|done|$ after classification, b is the number of `gqsort_partition` threadgroups, and t is the host time from encoding that iteration to the end of its read-back;

- once per sort: `sort n=<n> key=<key> wall_ms=<t> gpu_ms=<g> phase1_iterations=<i> phase1_sequences=<s> phase2_partitions=<p> altsorts=<a> max_stack_depth=<m>`.

Key values are never printed.

- **Configuration precedence.** Explicit flag or `Parameters` field, then `optp` default (K-05), then clamping (K-04). Clamping applies only to defaulted values. An explicit value that violates K-04 is an error, never silently changed.

There is no GUI surface, so no reference images apply.

6. Invariants (must hold in every valid implementation)

ID	Invariant
I-001	Sortedness. After a successful sort, $u_i \leq u_{i+1}$ for all $0 \leq i < n - 1$, where u is the C-04 code of the caller’s storage.
I-002	Permutation. After a successful sort, the multiset of 32-bit patterns in the caller’s storage equals the input multiset.
I-003	Determinism of output. For a given input, the output bytes are identical across runs, parameter choices and devices, and equal to <code>CPUReference</code> (they are fully determined by I-001 and I-002). Intermediate buffer contents and phase-one child order MAY vary between runs because of atomic ordering.
I-004	Gap soundness. Every index filled with pivot p (R-06) held, in its sequence, an element whose code equals p , and the number of filled indices equals the number of such elements. This requires codes to be exactly the bit patterns (C-04), which is why key–value sorting is a non-goal.
I-005	Disjointness and progress. At every point, the live sequences (<code>work</code> , <code>done</code> , phase-two stacks) are pairwise disjoint index ranges, disjoint from all finalized indices, and each child is strictly shorter than its parent.
I-006	No side effects on failure before the GPU starts. An error thrown during validation leaves the caller buffer byte-identical.
I-007	Ordering only through atomics, barriers, or dispatch boundaries. No kernel reads data written by another threadgroup of the same dispatch except atomic return values (R-09, R-10). Within a threadgroup, a thread reads device or threadgroup memory written by another thread only after a barrier covering that memory space (R-28).
I-008	Finalization in D. Each index $i \in [0, n)$ of D receives its final value exactly once, from a gap fill or an alternative-sort write-back. A partition write to D is never final.

7. Constraints (precise and measurable)

ID	Constraint
K-01	$0 \leq n \leq \text{maxKeys}$, where $\text{maxKeys} = \min(2^{31} - 1, \lfloor \text{maxBufferLength}/4 \rfloor)$.
K-02	Platform: macOS 15 or later on Apple silicon, with a device that supports <code>MTLGPUPFamily.apple7</code> (D-01). Swift 6 toolchain.
K-03	Threadgroup memory used by each pipeline MUST be $\leq \text{maxThreadgroupMemoryLength}$. Phase one needs at most $(4T + 16) \cdot 4$ bytes, and phase two at most $(\max(2T, \text{minseq}) + 3 \cdot 32 + 8) \cdot 4$ bytes; since minseq is a power of two, it also bounds the padded alternative-sort length of R-15 (compare [P Tab I]).

ID	Constraint
K-04	Validity: T is a power of two with $32 \leq T \leq \min(1024, \text{maxThreadsPerThreadgroup})$; $1 \leq \text{maxseq} \leq 2^{16}$; minseq is a power of two with $64 \leq \text{minseq}$ and satisfies K-03; $1 \leq \text{maxPhaseOneIterations} \leq 1024$. A defaulted value is clamped into its valid range (for a power-of-two parameter, to the nearest valid power of two), in this order: T first, then maxseq , then minseq (whose K-03 bound depends on T). An explicit invalid value throws <code>invalidParameters</code> . $\text{maxseq} = 1$ is valid and disables phase one (R-03).
K-05	Default parameters. With $s = n$, $\text{optp}(s, k, m) = 2^{\lfloor \log_2(sk+m) + 0.5 \rfloor}$ [P §5.3]. Defaults: $T = \text{optp}(n, k_T, m_T)$, $\text{maxseq} = \text{optp}(n, k_M, m_M)$, $\text{minseq} = \text{optp}(n, k_S, m_S)$, with the constants from the C-10 entry in effect for the device (D-06), then clamped per K-04. The paper’s 8800GTX constants (0.00001172, 53; 0.00003748, 476; 0.00004685, 211) are no longer defaults. They remain valid inputs for tests (T-20).
K-06	Phase-one splitting: $\text{minlength} = \lceil n / \text{maxseq} \rceil$ and $\text{blocksize} = \max\left(T, \left\lceil \sum_{w \in \text{work}} \ w\ / \text{maxseq} \right\rceil\right)$.
K-07	Phase one performs at most $\text{maxPhaseOneIterations}$ (default 64) iterations. <code>phaseOneCapReached</code> is <code>true</code> iff $\text{phaseOneIterations} = \text{maxPhaseOneIterations}$ and the R-08 loop condition still holds after that iteration; in that case <code>work</code> is merged into <code>done</code> (D-08). If the loop condition becomes false exactly at the last permitted iteration, <code>phaseOneCapReached</code> is <code>false</code> .
K-08	Phase-two stack capacity is 32 entries. Depth is counted as in C-02 <code>maxStackDepth</code> . With R-13, the depth for a sequence of length ℓ is at most $\lceil \log_2(\ell / \text{minseq}) \rceil + 2 \leq 27$ under K-01 and K-04, so the capacity is never reached by a correct implementation.
K-09	Space [P Thm 2], for the buffer API: the auxiliary buffer is exactly $4n$ bytes (<code>auxiliaryBytes</code> = $4n$), and <code>bookkeepingBytes</code> obeys the bound in §7.1. The auxiliary buffer MAY be cached and reused across calls when it is large enough; a cached buffer larger than $4n$ still reports $4n$ as used, and its excess is not counted. The <code>inout</code> [K] API additionally allocates one $4n$ -byte shared staging buffer, which is not counted in either field.
K-10	Complexity [P Thm 1]: on the <code>zero</code> distribution with $n \geq \text{minseq}$ and $\text{maxseq} \geq 2$, phase one performs exactly 1 iteration, no <code>lqsort</code> dispatch occurs (E-24), and phase two performs 0 partitions and 0 alternative sorts, i.e. $O(n)$ [P §5.4].
K-11	Timing: <code>wallTime</code> is measured with <code>ContinuousClock</code> from entry of <code>sort</code> (after validation) to return. <code>gpuTime</code> is the sum of <code>gpuEndTime</code> – <code>gpuStartTime</code> over the command buffers. Units: seconds in the API, milliseconds with 3 decimals in the CLI. Throughput is n/t in Mkeys/s; for $n = 0$ or $t = 0$ it is reported as 0.
K-12	CLI exit codes are exactly those of the table in §7.2. Every C-07 case and every CLI failure condition maps to exactly one code.
K-13	Performance (recorded, not gating): on the reference machine, for <code>uniform uint32</code> at $n = 2^{24}$ with defaults, the median <code>wall_ms</code> of GPU-Quicksort SHOULD be at most half that of the fastest of <code>cpu-swift</code> , <code>cpu-qsort</code> , <code>cpu-stdsort</code> and <code>cpu-stdsort-par</code> (compare [P §5.4] “twice the speed or more”).
K-14	<code>tune</code> with its default arguments MUST finish within 30 minutes on the reference machine. Configurations invalid on the device are skipped, not reported as errors.

7.1 Bookkeeping bound (K-09)

Let M be the resolved `maxseq`. Each phase-one iteration starts with $|\text{work}| < M$ sequences (40-byte `SequenceRecord` each) and dispatches at most $M + |\text{work}| < 2M$ threadgroups (16-byte `BlockDescriptor` each). Phase two receives $|\text{done}| < 2M$ sequences (16-byte `SortSequence` plus 16-byte `SortStats` each). The constant term covers statistics and argument buffers:

$$\text{bookkeepingBytes} \leq 40M + 16 \cdot 2M + 32 \cdot 2M + 2^{16} = 136M + 2^{16}$$

For $n \leq 1$ (E-01, E-02) both `auxiliaryBytes` and `bookkeepingBytes` are 0. Buffers allocated only under `GPUQS_TEST_HOOKS` (instrumentation for T-12..T-16, T-40..T-42) are excluded from `bookkeepingBytes`.

7.2 CLI exit codes (K-12)

Exit	Conditions
0	success
1	a verification failed in <code>verify</code> , <code>bench</code> or <code>tune</code> (E-22)
2	argument parsing error; <code>invalidParameters</code> (E-05); <code>--n</code> above <code>maxKeys</code> in <code>verify</code> , <code>bench</code> or <code>tune</code> , or above $2^{31} - 1$ in <code>gen</code> (which never creates a Metal device); <code>bench</code> or <code>tune</code> from a debug build without <code>--allow-debug</code>
3	<code>noMetalDevice</code> , <code>unsupportedDevice</code> , <code>shaderLibraryMissing</code> , <code>shaderLibraryLoadFailed</code> , <code>tunedParametersInvalid</code> (E-18, E-20)
4	unreadable or unwritable file; input size not a multiple of 4 (E-15); input file holding more than <code>maxKeys</code> keys (<code>tooManyKeys</code>); table write failure (E-23); invalid existing table under <code>tune --write</code> (E-25)
5	<code>allocationFailed</code> , <code>gpuExecutionFailed</code> , <code>internalInvariantViolated</code> ; <code>bufferNotShared</code> or <code>bufferTooSmall</code> (which the CLI cannot trigger with its own buffers, so they indicate an internal defect)

8. Edge cases and failure semantics

ID	Case	Semantics
E-01	$n = 0$	Returns immediately with a report of all zeros. No GPU work and no allocation. The buffer MAY be any length, including 0.
E-02	$n = 1$	Same as E-01 except <code>count = 1</code> . The buffer is untouched.
E-03	$2 \leq n < \text{minseq}$	Phase one is skipped (<code>phaseOneIterations = 0</code>). One <code>lqsort</code> threadgroup alternative-sorts $[0, n)$ from D .
E-04	All keys equal (<code>zero</code>), or long runs of equal keys	Equal keys become gap fills (R-06). K-10 holds for all-equal input. Runs of duplicates MUST NOT cause non-termination, because I-005 guarantees progress.
E-05	Explicit parameter violating K-04 or K-03	Throws <code>invalidParameters("<name>: <reason>")</code> before any GPU work. The CLI exits 2.
E-06	<code>buffer.storageMode != .shared</code>	Throws <code>bufferNotShared</code> . (<code>.private</code> is not supported: D-04.)
E-07	<code>buffer.length < 4 * count</code>	Throws <code>bufferTooSmall(required: 4*count, actual: length)</code> .
E-08	<code>count > maxKeys</code> or <code>count < 0</code>	Throws <code>tooManyKeys</code> . For a negative count, the thrown value reports count as given.
E-09	A command buffer completes with <code>.error</code>	Throws <code>gpuExecutionFailed(error.localizedDescription)</code> . The caller buffer contents are unspecified (possibly partially sorted or encoded). The instance stays usable for later calls.
E-10	A kernel sets a <code>SortStats.error</code> flag, or a read-back violates C-05 (e.g. <code>lnext > gnext</code> , or a cursor outside <code>[start, end]</code>)	Throws <code>internalInvariantViolated</code> with the sequence index and the values. The CLI exits 5.

ID	Case	Semantics
E-11	Buffer API called with a <code>KeyType</code> that differs from how the caller wrote the bytes	Not detectable. The keys are sorted by the declared type's order, and I-002 still holds. The generic array API cannot hit this case, because <code>GPUSortableKey</code> has only the three conformances in C-01 and callers MUST NOT add others (the protocol is documented as closed).
E-12	Allocation of <i>A</i> or of a descriptor buffer fails	Throws <code>allocationFailed(bytes:)</code> . The caller buffer is untouched (I-006).
E-13	Adversarial input that makes median-of-three pivots lopsided	Stays correct. Phase one is bounded by K-07. Phase two may degrade to $O(\ell^2)$ time per sequence, which is accepted (non-goal). Stack depth still obeys K-08.
E-14	Float NaNs, ± 0 , $\pm \infty$, subnormals	Ordered per C-04. Every bit pattern is preserved (I-002).
E-15	CLI <code>sort --in</code> file whose size is not a multiple of 4, or unreadable	Exit 4 with message <code>input size is not a multiple of 4</code> or the OS error text.
E-16	Concurrent <code>sort</code> calls on one instance	Serialized (C-01). Each gets a correct result and its own report.
E-17	Phase-one child of length 0	Discarded, not added to <code>work</code> or <code>done</code> .
E-18	<code>GPUQuickSort.metallib</code> missing from the bundle, or fails to load or lacks a kernel	<code>init</code> throws <code>shaderLibraryMissing</code> or <code>shaderLibraryLoadFailed</code> . The CLI exits 3 with <code>gpuqsort: error: shader library ...</code>
E-19	Shipped <code>.metallib</code> stale (stamp does not match the current source)	Not detected at runtime. The test suite fails (T-35).
E-20	<code>TunedParameters.json</code> missing, malformed, or fails C-10 validation	<code>init</code> throws <code>tunedParametersInvalid(<reason>)</code> . The CLI exits 3.
E-21	Device name has no entry in C-10	Uses <code>apple-default</code> . <code>info</code> shows <code>tuning: apple-default</code> (no exact entry for <code><name></code>).
E-22	<code>tune</code> hits a verification failure in any run	Aborts with exit 1, naming the configuration and size, and does not write the table even if <code>--write</code> was given.
E-23	<code>tune --write</code> cannot write the table file	Exit 4 with the OS error text. The existing file is left unchanged (write to a temp file, then atomic rename).
E-24	<code>done</code> is empty after phase one (every element was finalized by gap fills, e.g. the <code>zero</code> distribution)	No <code>lqsort</code> command buffer is encoded or committed; the sort proceeds to Decoding; <code>phaseOneSequences = 0</code> , <code>phaseTwoPartitions = 0</code> , <code>phaseTwoAltSorts = 0</code> , <code>maxStackDepth = 0</code> .
E-25	<code>tune --write</code> with a table file that exists but is not a valid C-10 table	Exit 4 with <code>table <path> is not a valid tuned-parameter table: <reason></code> before any measurement; the file is unchanged (C-10 <code>--write</code> rule 3).

9. Acceptance criteria, tests, and evals

Tests use swift-testing and run with `swift test` (debug configuration, so test hooks are present, D-21) on a machine that meets K-02. Tests marked (*recorded*) store their result in `SPEC_BUILD_REPORT.md`: performance and tuning runs (T-32..T-34) in §Performance, and the code inspection (T-17) in §Inspection. A suite test checks that each recorded section exists. A recorded test whose record does not exist yet is declared `.disabled(if: ...)` with a reason naming its ids. Such a skip means those ids are *verification pending*, and a build report MUST NOT claim conformance while any test is skipped for that reason.

9.1 Correctness (deterministic, GPU)

ID	Test
T-01	For every distribution in C-08 (including <code>fullrange</code>), every key type, and $n \in \{2, 3, 31, 32, 33, 63, 64, 65, 255, 256, 257, 1023, 1024, 1025, 4097, 65535, 65536, 65537, 10^6, 2^6, 2^7, 2^8, 2^9, 2^{10}, 2^{11}, 2^{12}, 2^{13}, 2^{14}, 2^{15}, 2^{16}, 2^{17}, 2^{18}, 2^{19}, 2^{20}, 2^{21}, 2^{22}, 2^{23}, 2^{24}\}$, seed 42: output bytes equal CPUReference (sort the C-04 codes with <code>Array.sort()</code> , then decode) byte for byte. Proves R-01 , R-02 , R-03 , I-001 , I-002 , I-003 .
T-02	Same as T-01 for $n = 2^{24}$ on <code>uniform</code> , <code>sorted</code> and <code>zero</code> , <code>uint32</code> . Proves R-08 , R-12 at scale.
T-03	Parameter grid: $T \in \{32, 64, 128, 256, 512, 1024\}$ (only values valid on the device), $maxseq \in \{1, 7, 64, 1024\}$, $minseq \in \{64, 256, 1024, \text{max valid}\}$, both pivot strategies, on <code>uniform</code> and <code>staggered</code> with $n = 300,001$: all outputs are identical to CPUReference. Proves R-16 , O-2 , I-003 , K-04 .
T-04	Run the same input 20 times with defaults: all outputs are identical. Proves I-003 .
T-05	Unit test on the CPU and a GPU round trip over a sample of 16 M patterns including all special classes: <code>decode(encode(b)) == b</code> , and <code>encode</code> is monotone with respect to <code>totalOrder</code> for pairs drawn from a list of 64 hand-picked floats (NaNs with payloads, ± 0 , $\pm \infty$, min subnormal, <code>.greatestFiniteMagnitude</code>). Proves C-04 , R-17 , E-14 .
T-06	$n = 0$ and $n = 1$: no command buffer is created (checked through a test hook counting commits), the report matches E-01 / E-02 , and the buffer is unchanged. Proves E-01 , E-02 .
T-07	$n = minseq - 1$: <code>phaseOneIterations == 0</code> , <code>phaseTwoAltSorts == 1</code> , and the output is correct. Proves E-03 .
T-08	<code>zero</code> with $n = 2^{20}$ and defaults: <code>phaseOneIterations == 1</code> , <code>phaseOneSequences == 0</code> , <code>phaseTwoPartitions == 0</code> , <code>phaseTwoAltSorts == 0</code> , no <code>lqsort</code> dispatch was committed (test hook counting dispatches per kernel), and the output is correct. Proves K-10 , E-04 , E-24 , R-06 .
T-09	Many duplicates: keys $k \% 3$ for $n = 10^6$, and keys where half are equal: output correct, run terminates. Proves E-04 , I-004 , I-005 .
T-10	Iteration cap: <code>uniform</code> with $n = 2^{20}$, $maxseq = 1024$ and <code>maxPhaseOneIterations = 1</code> : <code>phaseOneIterations == 1</code> , <code>phaseOneCapReached == true</code> , <code>phaseOneSequences == 2</code> minus the number of empty children, and the output is correct. With <code>maxPhaseOneIterations = 64</code> on the same input, <code>phaseOneCapReached == false</code> . Proves K-07 , R-08 .
T-11	For <code>uniform</code> $n = 2^{22}$ with $minseq = 64$, and for <code>sorted</code> : <code>maxStackDepth</code> (counted per C-02) $\leq \lceil \log_2(\ell_{\max}/64) \rceil + 2$, where $\ell_{\max}$ is the longest phase-two input sequence (exposed through a test hook). Proves R-13 , K-08 .
T-12	Fault injection (test hooks, D-21): force the stack capacity to 2 and check that <code>internalInvariantViolated</code> is thrown and that the next sort on the same instance succeeds. Separately, corrupt a <code>SequenceRecord</code> cursor after a phase-one dispatch (hook) and check that the read-back check throws <code>internalInvariantViolated</code> . Proves E-10 .

ID	Test
T-40	Phase-two adversarial inputs, with $maxseq = 1$ (so a single <code>lqsort</code> threadgroup handles the whole input, R-03) and $minseq = 64$, $n = 2^{14}$: organ-pipe $(0, 1, \dots, n/2 - 1, n/2 - 1, \dots, 0)$, sawtooth $(k \bmod 257)$, a median-of-3 killer for this pivot rule, built on the CPU by simulating R-14 on index positions (repeatedly assign the next-smallest value to the position of the median-of-three sample's second-smallest slot of the current range, then recurse on the larger side) with the result recorded as a fixture, and reverse-sorted input. Each output equals <code>CPUReference</code> , and $maxStackDepth \leq 27$ (K-08). Proves E-13 , K-08 , R-13 .
T-41	O-2 : with <code>phaseOnePivot = .minMaxAverage</code> , on <code>fullrange</code> for <code>int32</code> and <code>float32</code> (codes above 2^{31} exercise the overflow-free formula), <code>zero</code> , <code>sorted</code> and <code>uniform</code> at $n = 2^{20}$: outputs equal <code>CPUReference</code> ; a test hook checks that every phase-one child is strictly shorter than its parent (I-005) and that each child's pivot equals $lo + \lfloor (hi - lo)/2 \rfloor$ computed on the CPU from the child's actual contents; for <code>uniform</code> , <code>phaseOneCapReached == false</code> . Proves O-2 , I-005 .
T-42	E-09 fault injection: a test hook in the command runner reports the k -th committed command buffer as failed with a synthetic <code>NSError</code> , for $k \in \{1, 2, last\}$ of a <code>uniform</code> $n = 2^{20}$ sort. <code>sort</code> throws <code>gpuExecutionFailed</code> carrying the error's description; the next sort on the same instance succeeds. The CLI (debug build, environment variable <code>GPUQS_TEST_FAIL_CB=<k></code>) exits 5 with <code>gpuqsor: error: gpuExecutionFailed</code> Proves E-09 , K-12 .

9.2 Structure and memory model (deterministic, GPU + instrumentation)

ID	Test
T-13	Test hook recording per-iteration data: for <code>uniform</code> $n = 2^{22}$, check that each iteration's number of threadgroups equals $\sum \lceil \ell / blocksize \rceil$ with $blocksize$ from K-06 , that every child is in buffer $1 - src$, and that each iteration's children satisfy I-005 (disjoint, strictly shorter). Proves R-07 , R-08 , K-06 , I-005 .
T-14	Test hook counting atomic operations: <code>gqsort_partition</code> performs exactly 2 device-atomic read-modify-writes per threadgroup under <code>medianOfThree</code> , and exactly 6 (2 plus one each for <code>lmin</code> , <code>lmax</code> , <code>gmin</code> , <code>gmax</code>) under <code>minMaxAverage</code> (counted with a debug counter buffer). Proves R-09 , O-2 .
T-15	After every phase-one iteration (test hook reading D): every gap $[lnext, gnext)$ equals the pivot, and the count of pivot-equal elements in the parent input equals the gap length. Proves R-06 , R-10 , I-004 .
T-16	Instrumented run that records each index's finalization writes to D (debug counter buffer, <code>GPUQS_TEST_HOOKS</code>): every index is finalized exactly once. Proves I-008 , R-15 .

ID	Test
T-17 (<i>recorded</i>)	Code inspection, recorded in <code>SPEC_BUILD_REPORT.md</code> §Inspection as a checklist: both kernels read with stride T starting at $b + t$ (R-05); pass 2 skips pivot-equal elements (R-04); no kernel reads non-atomic data written by another threadgroup of the same dispatch (I-007); every R-28 barrier is present, at the gap fill, after each partition before the next pop, and around the alternative sort’s load and write-back; the stack pushes the longer child first (R-13); no kernel assumes a SIMD width of 32. The suite test checks that the checklist section exists and lists every item. Proves R-04, R-05, R-13, R-28, I-007.
T-31	The <code>MemoryLayout</code> size, stride and field offsets of the Swift mirrors of C-05/C-06 equal the MSL values, reported by a kernel that writes <code>sizeof/offsetof</code> into a buffer. Proves C-05, C-06.

9.3 API, validation and errors

ID	Test
T-18	Explicit $T = 48$, $T = 2048$, $minseq = 100$, $minseq$ exceeding K-03, $maxseq = 0$ and <code>maxPhaseOneIterations = 0</code> each throw <code>invalidParameters</code> naming the parameter, and the buffer is byte-identical afterwards. Proves K-04, K-03, E-05, I-006.
T-19	A <code>.private</code> buffer throws <code>bufferNotShared</code> . A buffer of $4n - 1$ bytes throws <code>bufferTooSmall</code> . <code>count = -1</code> and <code>count = maxKeys + 1</code> throw <code>tooManyKeys</code> . Buffers are untouched. Proves E-06, E-07, E-08, K-01, I-006.
T-20	With <code>TuningSource.constants(.paper8800GTX)</code> , <code>resolvedParameters(for:)</code> returns (64, 512, 256) for $n = 2^{20}$ and (256, 1024, 1024) for $n = 2^{24}$ when the device allows them, and applies the K-04 clamps, in K-04’s order, for $n = 0$ and $n = maxKeys$. With <code>.bundled</code> , the result equals <code>optp</code> computed from <code>GPUQuicksort.tuning</code> . Proves K-05, R-16, C-10.
T-21	The report for uniform $n = 2^{20}$ has <code>auxiliaryBytes == 4n</code> , <code>bookkeepingBytes</code> within the §7.1 bound for the resolved M (and also for $M = 1$ and $M = 2^{16}$), <code>count == n</code> , <code>wallTime > 0</code> , <code>gpuTime > 0</code> , $gpuTime \leq wallTime$, <code>parameters</code> equal to <code>resolvedParameters</code> , and <code>libraryVersion</code> , <code>metallibSHA256</code> and <code>tuningEntry</code> equal to <code>GPUQuicksort.version</code> , <code>GPUQuicksort.metallibSHA256</code> and <code>GPUQuicksort.tuning.entry</code> . For $n = 1$ both byte counts are 0. Proves R-21, C-02, K-09, K-11.
T-22	Eight concurrent tasks sort distinct arrays on one instance: all are correct. Proves E-16, C-01.
T-23	<code>sort(&[Float])</code> and <code>sort(&[Int32])</code> on <code>fullrange</code> $n = 10^5$ give the same result as the buffer API. Proves C-01, R-17.
T-24	Allocation failure injected through a test hook allocator throws <code>allocationFailed</code> , and the buffer is unchanged. Proves E-12.

9.4 Generators and CLI (integration)

ID	Test
T-25	Golden values: for each C-08 distribution with $n = 1024$ and seed 42, the SHA-256 of the <code>gen</code> output equals the value committed in <code>Tests/Fixtures/golden.json</code> . The fixture is produced once from an independent Python reference script checked in with it, which implements C-08 as written (including the 64-bit <code>gaussian</code> sum and the draw order $4k..4k + 3$). Also: MT19937 seeded with 5489 gives 3499211612 as its first output. Proves R-20, C-08.
T-26	Distribution properties for $n = 2^{20}$: every value is $< 2^{31}$; <code>sorted</code> is non-decreasing; <code>zero</code> is constant; <code>bucket</code> values of section S lie in $[Sw, (S + 1)w)$; <code>staggered</code> block i lies in the D-07 range; the <code>gaussian</code> mean is within 1% of 2^{30} . Proves C-08, D-07.
T-27	CLI: <code>gen</code> then <code>sort</code> then compare with <code>verify</code> 's oracle gives exit 0. <code>sort</code> on a 4097-byte file exits 4 with the E-15 message. <code>--threads 48</code> exits 2. Missing <code>--dist</code> exits 2. <code>gen --n</code> above <code>maxKeys</code> exits 2. <code>--table</code> pointing at an invalid table exits 3. <code>sort --out</code> into a read-only directory exits 4. Each case matches its §7.2 row. Proves R-19, K-12, E-15.
T-28	<code>bench --n 1M --runs 3 --dist uniform --cpu --allow-debug</code> (the test build is debug, D-21) emits the exact §5.2 header, 3×5 data rows (one <code>gpu-quicksort</code> and four <code>cpu-*</code> algorithms; the warm-up run is not emitted), <code>verified=true</code> on every row, provenance columns equal to <code>info --json</code> 's <code>version</code> and <code>metallibSHA256</code> and the tuning entry in effect, and nothing on <code>stderr</code> without <code>--verbose</code> . Proves R-23, R-22, R-26, §5.2.
T-29	Library: a <code>diagnostics</code> handler receives, per sort, <code>phaseOneIterations</code> lines matching the §5.3 <code>phase1</code> format with $i = 1, 2, \dots$, then exactly one line matching the <code>sort</code> format whose fields equal the returned <code>SortReport</code> . CLI: <code>--verbose</code> writes the same lines to <code>stderr</code> . No line contains a key value (checked with the <code>zero</code> distribution's constant c , which MUST NOT appear). Proves R-22, C-01.
T-30	<code>info --json</code> decodes as the §5.2 object: <code>limits.maxKeys</code> follows K-01, <code>tuning</code> equals <code>GPUQuicksort.tuning</code> , <code>metallibSHA256</code> equals the stamp file, and <code>defaults</code> equals <code>resolvedParameters</code> for both sizes. The human output contains the same values. Proves C-03, C-10, K-01.
T-35	Stamp check: the SHA-256 of <code>Sources/GPUQuicksort/Metal/GPUQuicksort.metal</code> followed by <code>Sources/CShared/include/SharedTypes.h</code> equals <code>Resources/metallib.sha256</code> , and both <code>.metallib</code> resources exist and load with <code>makeLibrary(URL:)</code> exposing all five kernel names. Proves R-27, C-09, E-19.
T-36	<code>scripts/build-metallib.sh</code> run with <code>OUT_DIR</code> set to a temporary directory (C-09) exits 0 and produces loadable libraries plus a stamp equal to the checked-in one. When run on a copy of the source with a syntax error injected, it exits non-zero and leaves the previous outputs unchanged. Skipped with a message when <code>xcrun metal</code> is unavailable. Proves C-09, R-27.

ID	Test
T-37	C-10 loader: the shipped table validates with <code>.bundled</code> ; tables passed with <code>.file(URL)</code> that lack <code>apple-default</code> , contain $k < 0$ or $m < 1$, contain a <code>sameAs</code> chain, or are malformed JSON each make <code>init</code> throw <code>tunedParametersInvalid</code> , as does <code>.constants</code> with $m = 0$; with a table lacking the host device’s name, <code>tuning.entry</code> is <code>apple-default</code> ’s target and <code>tuning.exactMatch == false</code> . Fit unit tests: points (1, 3), (2, 5), (3, 7) give $(k, m) = (2, 1)$; decreasing data gives $k = 0$, $m = \bar{x}$; $J = 1$ gives $(0, x_1)$. The CLI exits 3 on an invalid table. Proves C-10 , E-20 , E-21 .
T-38	<code>tune --n 64K,128K --runs 1 --grid small --allow-debug</code> (reduced grid test flag): (a) without <code>--write</code> , with <code>--table</code> pointing at an invalid file, it succeeds (the table is never read) and emits JSON matching §5.2; it computes the CPU oracle exactly once per size (hook counter); (b) with <code>--write --table <tmp></code> where <code><tmp></code> is a valid table, only the host device’s entry changes and the result validates; (c) with <code>--write</code> and a missing <code><tmp></code> , the file is created with <code>apple-default</code> pointing at the new entry and validates; (d) with <code>--write</code> and an invalid <code><tmp></code> , it exits 4 before measuring (hook: zero sorts performed) and the file is byte-identical; (e) an injected verification failure aborts with exit 1 and leaves the table unchanged; (f) an unwritable path exits 4; (g) without <code>--allow-debug</code> it exits 2. Proves R-24 , C-10 , E-22 , E-23 , E-25 .
T-39	<code>cpub_qsort_u32</code> , <code>cpub_stdsort_u32</code> , <code>cpub_stdsort_par_u32</code> and <code>cpu-swift</code> produce output identical to <code>CPUReference</code> for every C-08 distribution and key type at $n \in \{0, 1, 2, 1000, 10^6\}$. <code>bench</code> from a debug build exits 2 without <code>--allow-debug</code> . Proves R-26 , C-11 .

9.5 Performance (recorded)

ID	Test
T-32 <i>(recorded)</i>	<code>bench --dist all --n 1M,2M,4M,8M,16M --runs 5 --cpu</code> from a release build on the reference machine, after T-34 , recorded as a table analogous to [P Fig 4] with median <code>wall_ms</code> for all four algorithms. The K-13 ratio is reported for uniform 16M. The suite checks that the table exists. Proves K-13 .
T-33 <i>(recorded)</i>	Scaling check: median <code>wall_ms</code> from 1M to 16M uniform grows by a factor in [12, 24] (near-linear in $n \log n/p$, [P Thm 1]). The result is recorded. This is the empirical check of [P Thm 1] and has no gating id.
T-34 <i>(recorded)</i>	<code>gpuqsort tune --write --as-default</code> from a release build on the reference machine (Apple M5 Max): the full JSON output, the elapsed time (K-14), and the fitted (k, m) triples are recorded as the Apple analogue of [P Tab II] and [P Fig 11]. The suite checks that the recorded section exists, that the shipped C-10 table has an Apple M5 Max entry whose fitted date matches the recording, and that <code>apple-default</code> is <code>{"sameAs": "Apple M5 Max"}</code> . Before the first tuning run, the check is skipped with <code>.disabled(if: <no Apple M5 Max entry>, "tuning not yet recorded (R-25, T-34)")</code> , which makes R-25 <i>verification pending</i> (§9 intro). Proves R-24 , R-25 , K-14 .

10. Dependencies and environment

- **Toolchain:** Swift 6.0 or later (verified during drafting with 6.4), macOS 15 or later, Apple silicon (K-02). The Metal compiler (`xcrun metal`, `xcrun metallib`, from Xcode) is needed only to run `scripts/build-metallib.sh`. A plain swift build uses the checked-in `.metallib` files. A C++17 compiler (Apple clang) builds `CPUBaselines`.
- **Swift packages:** `apple/swift-argument-parser` from: "1.5.0" (CLI only). The library target has no third-party dependencies. It uses `Metal`, `Foundation` and `os`.
- **Tests:** `swift test` (swift-testing, debug configuration). GPU tests are skipped with a clear message when `MTLCreateSystemDefaultDevice()` returns `nil`. Test hooks (`GPUQS_TEST_HOOKS`) are compiled into debug builds of the library and CLI (C-09, D-21), and MUST NOT be present in release builds.
- **Golden fixtures:** `Tests/Fixtures/golden.json` plus `Tests/Fixtures/gen_reference.py` (Python 3, standard library only, own MT19937 implementation), used to produce T-25's hashes.
- **Build and run:** after editing `Sources/GPUQuickSort/Metal/GPUQuickSort.metal` or `Sources/CShared/include/Shared.h` run `scripts/build-metallib.sh` and commit its outputs. Then `swift build -c release` and `.build/release/gpuqsort info`. To tune: `.build/release/gpuqsort tune --write --as-default`, then commit `TunedParameters.json`.

11. Traceability matrix (id → where realized)

Spec id	Where realized (component)	Verified by
R-01	<code>GPUQuickSort.sort</code> , <code>KeyCodec</code>	T-01, T-23
R-02	whole pipeline	T-01, T-05
R-03	<code>Sorter</code> (host), <code>gqsort_*</code> , <code>lqsort</code>	T-01, T-07
R-04	<code>gqsort_partition</code> , <code>lqsort</code>	T-15, T-17
R-05	<code>gqsort_partition</code> , <code>lqsort</code>	T-17
R-06	<code>gqsort_fill</code> , <code>lqsort</code>	T-08, T-15
R-07	<code>Sorter</code> , descriptors	T-13
R-08	<code>Sorter.phaseOne</code>	T-02, T-13
R-09	<code>gqsort_partition</code>	T-14
R-10	<code>Sorter.phaseOne</code> , <code>gqsort_fill</code>	T-15
R-11	<code>Sorter.pickPivot</code>	T-03
R-12	<code>Sorter.phaseTwo</code> , <code>lqsort</code>	T-02, T-07
R-13	<code>lqsort</code> stack	T-11, T-17
R-14	<code>lqsort</code>	T-01, T-17
R-15	<code>lqsort</code> bitonic	T-07, T-16
R-16	<code>ParameterResolver</code>	T-03, T-20
R-17	<code>key_encode</code> , <code>key_decode</code> , <code>KeyCodec</code>	T-05, T-23
R-18	<code>GPUQuickSort</code> , <code>GPUQuickSortError</code>	T-18, T-19, T-24
R-19	<code>gpuqsort</code> CLI	T-27, T-28
R-20	<code>Distributions</code> , MT19937	T-25, T-26
R-21	<code>SortReport</code> assembly	T-21
R-22	<code>Diagnostics</code> (<code>os.Logger</code> + handler), CLI verbosity	T-28, T-29
R-23	<code>bench</code> command	T-28
R-24	CLI <code>tune</code> , <code>Tuner</code>	T-34, T-38
R-25	<code>Resources/TunedParameters.json</code>	T-34, T-37
R-26	<code>CPUBaselines</code> , <code>bench</code>	T-28, T-39
R-27	<code>scripts/build-metallib.sh</code> , <code>ShaderLibrary</code>	T-35, T-36
R-28	<code>lqsort</code> , <code>gqsort_partition</code> barriers	T-17, T-01
C-01	<code>GPUQuickSort</code> , <code>TuningSource</code>	T-20, T-22, T-23, T-29, T-37
C-02	<code>SortReport</code>	T-21
C-03	<code>Parameters</code> , <code>DeviceLimits</code>	T-20, T-30

Spec id	Where realized (component)	Verified by
C-04	KeyCodec, key_encode/decode	T-05
C-05	CShared/include/SharedTypes.h, gqsort_*	T-13, T-31
C-06	CShared/include/SharedTypes.h, lqsort	T-11, T-31
C-07	GPUQuicksortError	T-18, T-19, T-24
C-08	Distributions	T-25, T-26
C-09	Package.swift, scripts/build-metallib.sh, ShaderLibrary	T-35, T-36
C-10	TunedParameters, Tuner	T-20, T-37, T-38
C-11	CPUBaselines target	T-39
I-001	pipeline	T-01
I-002	pipeline	T-01, T-05
I-003	pipeline	T-03, T-04
I-004	gqsort_fill, lqsort	T-09, T-15
I-005	Sorter, lqsort	T-09, T-13, T-41
I-006	validation in GPUQuicksort.sort	T-18, T-19, T-24
I-007	all kernels	T-17, T-01
I-008	gqsort_fill, lqsort	T-16
K-01	DeviceLimits	T-19, T-30
K-02	GPUQuicksort.init	T-30 (runs only on a supported device)
K-03	ParameterResolver	T-18
K-04	ParameterResolver	T-03, T-18, T-20
K-05	ParameterResolver	T-20
K-06	Sorter.phaseOne	T-13
K-07	Sorter.phaseOne	T-10
K-08	lqsort	T-11, T-40
K-09	BufferPool	T-21
K-10	pipeline	T-08
K-11	SortReport, CLI	T-21, T-28
K-12	CLI exit mapping (§7.2)	T-27, T-42
K-13	pipeline	T-32
K-14	Tuner	T-34
E-01	GPUQuicksort.sort	T-06
E-02	GPUQuicksort.sort	T-06
E-03	Sorter	T-07
E-04	pipeline	T-08, T-09
E-05	ParameterResolver	T-18
E-06	validation	T-19
E-07	validation	T-19
E-08	validation	T-19
E-09	CommandRunner	T-42
E-10	Sorter read-back, lqsort	T-12
E-11	GPUSortableKey (closed set of conformances)	T-23
E-12	BufferPool	T-24
E-13	pipeline	T-40
E-14	KeyCodec	T-05, T-01
E-15	CLI sort	T-27
E-16	GPUQuicksort lock	T-22
E-17	Sorter.phaseOne	T-13
E-18	ShaderLibrary	T-35
E-19	test suite	T-35
E-20	TunedParameters loader	T-37

Spec id	Where realized (component)	Verified by
E-21	<code>TunedParameters</code> lookup	T-37
E-22	<code>Tuner</code>	T-38
E-23	<code>Tuner</code> writer	T-38
E-24	<code>Sorter</code>	T-08
E-25	<code>Tuner</code> table validation	T-38
O-1	retired in v0.2: <code>tune</code> is required (R-24)	—
O-2	<code>PhaseOnePivot.minMaxAverage</code> , <code>gqsort_partition</code> min/max reduction and atomics	T-03 , T-14 , T-41

12. Open questions and decisions to confirm

ID	Decision	Default taken	Alternatives	Affects	Owner / status
D-01	Target platform	macOS 15+, Apple silicon, apple7 family	include iOS/iPadOS; Intel Macs with AMD GPUs	K-02 , C-09 , R-01	requester / confirmed v0.2
D-02	Key types	<code>UInt32</code> , <code>Int32</code> , <code>Float</code> through an order-preserving code (C-04)	<code>UInt32</code> only; 64-bit keys; key-value pairs	R-01 , R-17 , C-04 , I-004 , E-14	requester / confirmed v0.2
D-03	Replace the paper’s “last block to finish” step ([P Alg 2]) with a follow-up <code>gqsort_fill</code> dispatch plus a host read-back	follow-up dispatch (Metal atomics are relaxed-only)	last-finisher with device-scope fences	R-10 , I-007 , C-05 , T-15	requester / confirmed v0.2
D-04	Buffer storage mode	<code>.shared</code> only	also <code>.private</code> via a staging blit	E-06 , C-01	requester / confirmed v0.2
D-05	Shader delivery	precompiled .metallib built by <code>scripts/build-metalSPM.sh</code> checked in with a source-hash stamp, loaded with <code>makeLibrary(URL:)</code>	runtime compile from source (the v0.1 default, rejected); an <code>SPM</code> build-tool plugin	R-27 , C-09 , C-07 , E-18 , E-19 , K-12 , T-35 , T-36	requester / confirmed v0.2 (changed)
D-06	Default <code>optp</code> constants	fitted on the Apple GPU by the required <code>tune</code> command (R-24) and shipped in C-10 ; the paper’s 8800GTX constants are used only in tests	the paper’s Table II rows (the v0.1 default, rejected)	K-05 , R-16 , R-24 , R-25 , C-10 , K-14 , T-20 , T-34 , T-37 , T-38	requester / confirmed v0.2 (changed)
D-07	Staggered formula	0-based i with $i < p/2$ (matches Helman et al. 1998)	1-based i with the paper’s formula, clamped	C-08 , T-26	requester / confirmed v0.2

ID	Decision	Default taken	Alternatives	Affects	Owner / status
D-08	Phase-one iteration cap	64, configurable; on cap, the remaining work goes to phase two	no cap; $2\lceil\log_2 n\rceil$	K-07 , E-13 , T-10	requester / confirmed v0.2
D-09	Concurrency model of one instance	serialize calls with an internal lock	non- <code>Sendable</code> class; parallel sorts	C-01 , E-16 , T-22	requester / confirmed v0.2
D-10	Default phase-one pivot	minMaxAverage (revised v0.5: on the reference machine it was faster on every [P §5.3] distribution at 16M and 64M — $1.08\text{--}1.57\times$, and $9.2\times$ on staggered 64M, where median-of-three needed 47 iterations and 440 ms; see <code>PERFORMANCE.md</code>)	<code>medianOfThree</code> (the v0.2–v0.4 default); random	R-11 , O-2 , T-03 , T-15 , T-41	requester / confirmed v0.5 (changed)
D-11	Pivot index one past the end in [P Alg 1, Alg 3]	use index $e - 1$	—	R-11 , R-14	requester / confirmed v0.2
D-12	CPU comparison	four baselines: Swift <code>Array.sort()</code> , <code>libc qsort</code> , <code>C++ std::sort</code> , and parallel <code>std::sort(std::execution::par)</code> , via a C-ABI shim (C-11); the correctness oracle stays Swift <code>Array.sort()</code>	three sequential baselines (v0.2–v0.4)	R-26 , C-11 , K-13 , T-28 , T-32 , T-39	requester / confirmed v0.5 (extended)
D-13	Prefix-sum implementation inside a threadgroup	free, as long as the result is correct	pin the Blelloch scan	R-04	requester / confirmed v0.2
D-14	Test framework	swift-testing	XCTest	§9, §10	requester / confirmed v0.2
D-15	Performance target K-13	SHOULD, recorded only	a gating MUST	K-13 , T-32	requester / confirmed v0.2

ID	Decision	Default taken	Alternatives	Affects	Owner / status
D-16	C-10 lookup and bootstrap	exact <code>MTLDevice.name</code> , else <code>apple-default</code> ; the table ships with a <code>paper-8800gtx</code> bootstrap entry as <code>apple-default</code> until the M5 Max fit replaces it (F-001)	refuse to run untuned; per-GPU-family keys	C-01 , C-10 , R-24 , R-25 , E-21 , T-20 , T-34 , T-37 , T-38	implementer / confirm
D-17	<code>tune</code> method details	full grid over valid powers of two, <code>--runs 3</code> median, ties to smaller values, OLS with $k \geq 0, m \geq 1$	the paper’s “best/worst/average” sweep [P Fig 10] only; fitting in $\log_2$ space	R-24 , C-10 , K-14 , T-38	implementer / confirm
D-18	Detecting a stale <code>.metallib</code>	SHA-256 stamp of <code>.metal + .h</code> checked by the test suite	compare file modification times; a CI job rebuilds and diffs the output	R-27 , E-19 , T-35 , T-36	implementer / confirm
D-19	CPU baselines’ timed region for <code>int32/float32</code>	includes the CPU C-04 encode/decode, to match the GPU’s end-to-end timing	sort native types directly (<code>Float</code> comparisons break on NaN)	C-11 , R-26	implementer / confirm
D-20	O-2 root pivot (the root’s min and max are unknown before any partition)	<code>medianOfThree</code> for the root, min/max-average afterwards	an extra <code>minmax_reduce</code> dispatch over $[0, n)$, matching [P §5.2] literally	O-2 , T-41	implementer / confirm
D-21	Where test hooks are compiled	debug configuration of the library and CLI (<code>.when(configuration: .debug)</code>), so <code>swift test</code> and CLI fault injection see them	a separate test-only target; runtime flags in release builds	C-09 , §10, T-12 , T-28 , T-38 , T-42	implementer / confirm
D-22	<code>maxseq = 1</code>	valid; disables phase one (R-03), which T-40 uses to drive phase two alone	raise K-04 ’s lower bound to 2	R-03 , K-04 , K-10 , T-03 , T-40	implementer / confirm

ID	Decision	Default taken	Alternatives	Affects	Owner / status
D-23	How to get a parallel <code>std::sort</code> on macOS	libc++’s parallel algorithms (libdispatch backend), enabled with <code>-fexperimental-library</code> on the <code>CPUBaselines</code> target; the linked <code>libc++experimental.a</code> in the current SDK is built for macOS 27, so the linker warns when targeting macOS 15, and the parallel baseline is a benchmark-only dependency	Intel oneTBB / <code>pstld</code> as the backend; a hand-written parallel merge sort (not <code>std::sort</code>)	R-26 , C-11 , T-39 , §10	implementer / confirm

Appendix A — Algorithm and complexity, restated (informative)

This appendix restates, in this specification’s own terms, the algorithm the normative rows (§2–§8) require and the complexity arguments behind them. It is **informative**: when it and a normative row disagree, the row wins. The pseudocode was written from this implementation (`Sorter.swift`, `GPUQuicksort.metal`), not transcribed from the paper; each step names the ids that pin it down and the paper passage the idea comes from. The paper is cited as [P ...] (Cederman and Tsigas 2009, [doi:10.1145/1498698.1564500](https://doi.org/10.1145/1498698.1564500)). Its text is not reproduced here, apart from three short expressions quoted in A.6 to show exactly where this design departs from it.

A.1 Notation

Symbol	Meaning
n	number of keys
D, A	the caller’s buffer and the auxiliary buffer, n codes each (§3.3)
code	the order-preserving <code>UInt32</code> form of a key (C-04); the kernels only ever compare codes
T	threads per threadgroup (a power of two, K-04)
M	<code>maxseq</code> , the phase-one sequence budget
S	<code>minseq</code> , the length below which a sequence is sorted by the alternative sort (R-15)
sequence	a half-open index range $[b, e)$ plus <code>src</code> $\in \{D, A\}$, the buffer that currently holds its elements (R-07)
$\overline{src}$	the other buffer
gap	the range between a partition’s <code><</code> part and <code>></code> part, where the pivot-equal elements go; its indices are final (R-06)

A.2 The whole sort

```

SORT(D, n, keyType)
  if n <= 1: return
  if keyType != uint32: ENCODE(D)           one pass, own dispatch
  A <- auxiliary buffer of n codes
  done <- PHASE-ONE(D, A, n)
  if done is not empty: PHASE-TWO(D, A, done)
  if keyType != uint32: DECODE(D)

```

R-01, R-03
E-01, E-02
R-17, C-04
K-09
R-08
R-12, E-24
R-17

Every index of D receives its final value exactly once: either from a gap fill (phase one or phase two) or from an alternative-sort write-back (I-008). Partition writes into D are never final.

A.3 Phase one: many threadgroups per sequence

The host loop ([P §3.2.1], [P Alg 1]):

```

PHASE-ONE(D, A, n)
  if n < S: return [ ([0, n), D) ]
  minlength <- ceil(n / M)
  work <- [ ([0, n), D) with pivot MEDIAN3 of D[0], D[n/2], D[n-1] ]
  done <- [ ]
  iteration <- 0
  while work is not empty and |work| + |done| < M:
    if iteration = maxIterations: capReached <- true; stop
    iteration <- iteration + 1
    blocksize <- max(T, ceil(total length of work / M))
    for each sequence w in work:
      record_w <- { start, end, lnext := start, gnext := end,
                    pivot, src, lmin/lmax/gmin/gmax := identities }
      cut w into sections of blocksize; the last takes the remainder
    in ONE command buffer:
      PARTITION: one threadgroup per section
      FILL:      one threadgroup per section (a later dispatch)
    wait for completion; read the records back
    for each w with record r:
      check start <= lnext <= gnext <= end
      lower <- [start, lnext) in other(src); upper <- [gnext, end) in other(src)
      drop empty children
      child pivot <- minMaxAverage(child) or MEDIAN3(child)
      child goes to done if |child| < minlength, else to work
  return done ++ work

```

R-08
E-03
K-06
D-20

R-08 (M = 1: no iteration, R-03)
K-07

K-06
C-05

R-10, D-03
R-04, R-05, R-09
R-06

E-10
E-17
R-11, 0-2

R-04

One threadgroup's partition of its section $[b_s, e_s)$ of sequence r , with pivot p (the kernel `gqsort_partition`):

```

PARTITION-SECTION(r, [b_s, e_s))
  pass 1: thread t counts lt_t = #{ v < p } and gt_t = #{ v > p }
           over indices b_s + t, b_s + t + T, b_s + t + 2T, ...
  exclusive prefix sums over threads: L_t, G_t; totals L, G
  thread 0: lowBase <- atomic fetch-add(r.lnext, L)
            highBase <- atomic fetch-sub(r.gnext, G) - G
  barrier: every thread learns lowBase, highBase
  pass 2: thread t re-reads the same indices and writes
           v < p to other(src)[lowBase + L_t + j]
           v > p to other(src)[highBase + G_t + j]
           v = p is not written

```

R-05 (coalesced)
R-04, D-13
R-09 (one atomic per side
per threadgroup)
R-28(a)

R-04

```

    if minMaxAverage: reduce this section's min and max of each side;
                        one atomic min/max per field into r
                                                                0-2

```

FILL, a separate dispatch, gives block j of a sequence the j -th slice of that sequence's gap $[l_{next}, g_{next})$ and writes p there in D . Because it is a later dispatch, it sees the final cursors after every partition threadgroup has finished (R-10, I-007).

A.4 Phase two: one threadgroup per sequence

([P §3.2.2], [P Alg 3]). The kernel `lqsort` runs one threadgroup per sequence in done:

```

PHASE-TWO-THREADGROUP(root)                                     R-12
  if 0 < |root| < S: ALT-SORT(root); stop                       R-15
  push root
  while the stack is not empty and no error:
    barrier (device + threadgroup)                             R-28(b)
    pop [b, e) in src                                           (the shorter part: R-13)
    p <- MEDIAN3(s[b], s[floor((b+e)/2)], s[e-1])              R-14, D-11
    two-pass partition of [b, e) into other(src), as in A.3, with
      lowBase = b and highBase = e - G (no atomics needed)     R-04, R-05
    barrier (device + threadgroup)                             R-28(a)
    fill D[b + L, e - G) with p                                 R-06
    children: [b, b + L) and [e - G, e) in other(src)
    push the longer child, then the shorter, if length >= S;
      a full stack sets the error flag                          R-13, K-08, E-10
    barrier; ALT-SORT each child with 0 < length < S           R-15, R-28(b)

ALT-SORT([b, e) in src)                                         R-15
  load the elements into threadgroup memory, pad with 0xFFFFFFFF up to a power of two
  barrier                                                       R-28(c)
  bitonic sorting network over the padded array
  write the first e - b elements to D[b, e)                    I-008

```

A.5 Pivot rules

Rule	Used by	Definition	Ids
median of three	phase-one root (both strategies); phase-one children under <code>medianOfThree</code> ; every phase-two partition	the median of the first, middle ($\lfloor (b + e)/2 \rfloor$) and last ($e - 1$) element of the range	R-11, R-14, D-11, D-20
min/max average (default)	phase-one children	$p = lo + \lfloor (hi - lo)/2 \rfloor$ over the child's minimum and maximum code, in unsigned 32-bit arithmetic	R-11, O-2, D-10

A.6 Where this design departs from the paper

Topic	The paper	This design	Why	Ids
Finishing a phase-one iteration	the last threadgroup to finish detects that it is last, fills the gap and derives the child sequences; the test is written “ <code>FAA(blockcount, -1) = 0</code> ” [P Alg 2]	a separate <code>FILL</code> dispatch fills the gap; the host reads the records back and derives the children	Metal’s atomics are relaxed-only, so a “last” threadgroup cannot safely read the other threadgroups’ results; also, fetch-and-add returns the old value, so with the counter initialized to the block count that test is off by one	R-10 , I-007 , D-03
Pivot sample	the last sample element is written “ <code>d_size</code> ” [P Alg 1] and “ <code>s_end</code> ” [P Alg 3], one past the end of the range	the last sample is index $e - 1$	out-of-range read	R-11 , R-14 , D-11
Phase-one pivot	the experiments use the average of the minimum and maximum [P §5.2]	the same rule, as the default since v0.5; the root uses median of three because its minimum and maximum are not yet known	median of three split staggered badly (about 47 iterations at 64M)	R-11 , O-2 , D-10 , D-20

Topic	The paper	This design	Why	Ids
Keys	integers and floats compared directly	every key is converted to an order-preserving <code>UInt32</code> code first; floats follow IEEE 754 <code>totalOrder</code>	one kernel path; the gap fill writes the pivot's exact bit pattern, which is only a permutation if equal codes mean equal bits	R-17 , C-04 , I-004
Phase-one budget	fixed per GPU from the paper's measurements [P Tab II]	fitted on the target GPU by <code>gpuqsort tune</code>	Apple GPUs favor different parameters	R-24 , C-10 , D-06

A.7 Complexity

Work per partition level. A level partitions each live element twice (a counting pass and a scatter pass) and runs one prefix sum per threadgroup of T threads, which costs $O(\log T)$. With p processors, a level therefore costs $O(n/p + \log T)$, and T does not depend on n .

Average time ([P Thm 1]). If pivots split sequences in a bounded ratio on average, the recursion has $O(\log n)$ levels. Sequences shorter than S are finished by a bitonic network of size at most S , a constant that does not grow with n . So the average time is

$$O\left(\frac{n}{p} \log n\right),$$

with p the number of processors. Adversarial inputs can still force $O(n^2)$ work under median-of-three; this spec does not guard against that (§0 non-goals, [E-13](#)).

A width bound for the default pivot. Under `minMaxAverage`, a child's codes lie within either $[lo, p]$ or $[p + 1, hi]$, and each of those ranges is at most half the parent's $[lo, hi]$ (rounded up). A 32-bit code range can be halved only 32 times before it holds a single value, and a sequence of equal codes is finished by one gap fill. So along any path in phase one, a sequence is partitioned at most 32 times after the root, whatever the input order: phase-one work is $O(\frac{n}{p} \cdot 32)$ in the worst case. Phase two still uses median of three, so this bound covers phase one only.

Progress. Every child is strictly shorter than its parent ([I-005](#)). Under median of three, the pivot is an element of the range, so the gap is non-empty. Under min/max average, $lo \leq p < hi$ whenever $lo < hi$, so the element with code hi leaves the $<$ side and the element with code lo leaves the $>$ side; if $lo = hi$ the whole sequence becomes a gap.

Stack depth (K-08). Phase two always pops the shorter child, and each stack entry below the top is at most as long as the part it was split from. So the k -th entry from the bottom has length at most about $\ell/2^{k-1}$, and an entry is pushed only if it has at least S elements. This gives a depth of at most $\lceil \log_2(\ell/S) \rceil + 2$, which is at most 27 under K-01 and K-04, so a 32-entry stack never overflows in a correct implementation.

Space ([P Thm 2]). The sort needs D and A , $2n$ codes, plus bookkeeping of at most $136M + 2^{16}$ bytes (§7.1). M does not depend on n beyond the fitted `optp` term and is capped at 2^{16} (K-04), so the total is $2n + c$.

Host round trips. Each phase-one iteration costs one command buffer and one read-back, so the fixed host overhead grows with the number of iterations, not with n . That is why small sorts are dominated by round trips and large ones by GPU work (PERFORMANCE.md, F-031).

Revision history

- **v0.5 (2026-09-25):** requester approved two changes after the 32M/64M measurements:
 - **D-10 revised:** the default phase-one pivot is now `minMaxAverage` (R-11, O-2, C-03, §5.2). Evidence: PERFORMANCE.md (faster on all six distributions; 9.2× on `staggered` 64M).
 - **D-12 extended:** a fourth CPU baseline, `cpu-stdsort-par` (R-26, C-11, K-13, T-28, T-39; D-23 for the toolchain caveat).
- **v0.4 (2026-09-25):** folded in round-2 review findings F-020..F-030:
 - **F-020:** T-28 and T-38 pass `--allow-debug`; `tune` gets the debug guard; recorded runs use release builds.
 - **F-021:** C-10 `--write` rules for a missing, valid or invalid table; E-25; T-38 split into cases (a)–(g).
 - **F-022:** skipped recorded tests mean *verification pending* (§9 intro); T-34 uses `.disabled(if:)`.
 - **F-023:** R-28 title.
 - **F-024:** `phaseOneCapReached` boundary (K-07, C-02).
 - **F-025:** test-hook buffers excluded from §7.1.
 - **F-026:** RFC 4180 CSV.
 - **F-027:** `diagnostics` setter locking.
 - **F-028:** `gen` bound is $2^{31} - 1$ and needs no Metal device.
 - **F-029:** `CShared.c`.
 - **F-030:** T-40 killer construction; build-report sections.
- **v0.3 (2026-09-25):** folded in SPEC_REVIEW_REPORT.md (v0.2 review), P0 through P2:
 - **F-001:** `TuningSource` added to C-01; bootstrap table `paper-8800gtx`; `tune` independent of the table (R-24, R-25, C-10, T-20/T-34/T-37/T-38; D-16 revised).
 - **F-002:** R-28 intra-threadgroup ordering; I-007 extended; T-17 checklist.
 - **F-003:** O-2 defined in §2.5; T-14 extended; T-41; D-20.
 - **F-004:** K-09 rewritten with the §7.1 bound; `bookkeepingBytes`; T-21.
 - **F-005:** R-03 allows `maxseq = 1`; K-10 conditioned on `maxseq ≥ 2`; D-22.
 - **F-006:** E-24 empty done; §3.1 transition; T-08.
 - **F-007:** C-01 `diagnostics` handler; §5.3 line formats; R-22; T-29.
 - **F-008:** `SharedTypes.h` moved to `CShared/include` with a `GQS_ATOMIC_U32` macro; `exclude: ["Metal"]`; script and stamp paths.
 - **F-009:** `gaussian` pinned; bucket cleaned.
 - **F-010:** T-10 is now a deterministic cap test; T-40 covers phase-two adversarial inputs.
 - **F-011:** T-42 command-buffer fault injection; T-12 narrowed to E-10; D-21.
 - **F-012:** §7.2 exit-code table; K-12 points to it; T-27.
 - **F-013:** oracle computed once per size in `bench/tune`.
 - **F-014:** provenance fields in `SortReport`, the bench CSV and the tune JSON; `GPUQuicksort.version`.
 - **F-015:** `keyTypeMismatch` removed.
 - **F-016:** R-23, K-03, K-04 clamp order, T-17 marker, `OUT_DIR`, `info --json` object.
 - **F-017:** allocation and read-back failure transitions.
 - **F-018:** stack-depth convention.
 - **F-019:** K-02 rationale dropped; K-09 scoped to the buffer API; `sort` is synchronous.

- **v0.2 (2026-09-25):** requester ratified [D-01..D-04](#), [D-07..D-11](#), [D-13..D-15](#). Changed [D-05](#) (precompiled `.metallib` via script: [R-27](#), [C-09](#) rewritten, [E-18/E-19](#), [T-35/T-36](#)), [D-06](#) (`tune` is now required and defaults come from Apple-fitted constants: [R-24/R-25](#), [C-10](#), [K-05](#) rewritten, [K-14](#), [E-20..E-23](#), [T-37/T-38](#); [O-1](#) retired), and [D-12](#) (three CPU baselines: [R-26](#), [C-11](#), [T-39](#); `bench` rows and [K-13](#) updated). Added [D-16..D-19](#) for new defaults.
- **v0.1 (2026-09-25):** first draft.